Vector Search

Multimedia Retrieval · Chapter 6

Multimedia Retrieval - 2026 - Uni Basel

Roger Weber

Contents

6 - Vector Search 1
Distance and Similarity Measures 3
Best-match search with similarity scores 3
Nearest-neighbor search with distances 4
Keeping dimensions comparable 6
Low-Dimensional Search Structures 8
Precomputing all answers: Voronoi diagrams 8
From B-trees to spatial indexes 8
R-trees and the problem of overlap 9
An optimal nearest-neighbor algorithm 10
Beyond Euclidean distance: dot product and cosine 13
A fundamental limit 13
The Curse of Dimensionality 15
Distances grow with the dimension 15
Distances lose their contrast 15
Consequence 1: trees can no longer prune 17
Consequence 2: what does “nearest” still mean? 19
The dot product: losing direction 19
Dot products concentrate around zero 20
Good matches instead of the best match 21
Approximate Nearest-Neighbor Search 22
Fewer dimensions 22
Fewer bits per vector 23
Fewer vectors read 26
Combining the levers 28
Measuring the trade-off 28
Why does approximate search work? 29
Vector Search in Practice 32
A running example: movie search 32
Vector search in four systems 32
Adding filters and keyword search 35
Choosing a system 38
Summary 40
Method Comparison 40
Key Takeaways 40
Key Formulas 41
Self-Check Questions 42
Further Reading 43

6 - Vector Search

The previous chapter turned text into embeddings: dense vectors whose geometry captures meaning, so that "heart disease" and "cardiology treatment" land close together even though they share no words. Images, audio, and video can be encoded the same way. Once every document and every query is a point in the same space, retrieval becomes a geometric question: which stored points lie closest to the query point? Figure 6.1 makes this concrete, showing a 2D projection of a word-embedding space in which related terms fall into the same neighborhood, so that proximity stands in for similarity in meaning.

That question is easy to state and expensive to answer. A collection may hold millions or billions of vectors, each with hundreds or thousands of dimensions. Comparing the query against every stored vector is correct but slow, and the classical spatial indexes that accelerate low-dimensional search collapse as the number of dimensions grows. That leaves a gap between the only method guaranteed to be correct, the full scan, and the speed a real system needs. This chapter is about closing it: how to measure closeness, why exact search becomes hopeless in high dimensions, and how approximate methods recover speed by giving up a little accuracy that the application never needed.

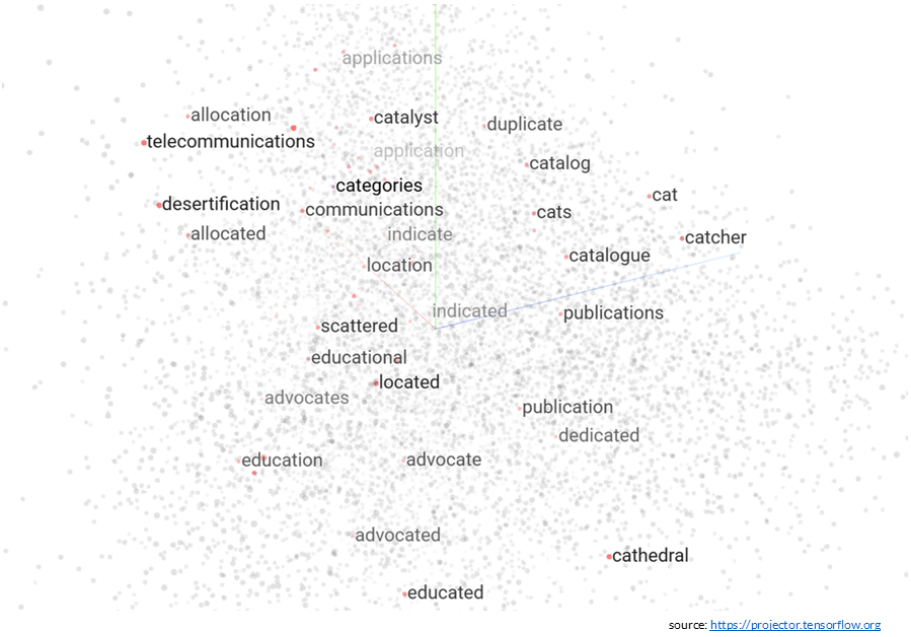


Figure 6.1: A 2D projection of a word-embedding space; semantically related terms form visible clusters.

The vectors themselves arrive from more than one source. We already have embeddings from learned models, built in the previous chapter; other applications extract vectors straight from the media, such as a color histogram of an image. Whatever their origin, the question this chapter asks is how to compare them, and there are two ways to do it. A similarity score, most often the dot product or cosine, rewards a close match with a larger value. A distance, such as Euclidean distance, marks a close match with a smaller value. The two are two sides of the same coin: a simple normalization turns one into the other, so the engineering problem, finding the closest points quickly, is the same whichever we use.

What you'll learn

- Explain how dot product, cosine similarity, and distance measures express closeness, and convert between them.
- Describe how classical spatial indexes (Voronoi diagrams, grid files, R-trees) answer nearest-neighbor queries.
- Explain the curse of dimensionality and why it forces exact search back to a full scan.

- Compare approximate nearest-neighbor methods (IVF, HNSW, LSH, product quantization) and their recall-speed trade-off.
- Choose a vector-search framework and combine similarity search with metadata filters.

Prerequisites

- Semantic Search: how models place meaning in a vector space. See Word Embeddings and Building a Semantic Search System.
- Performance Evaluation: recall, which reappears here as the quality axis of approximate search.
- Basic linear algebra: vectors, dot products, norms, and eigenvectors.

Distance and Similarity Measures

Consider again the query “heart disease”. In Vector Space Retrieval, the query and every document became TF-IDF vectors. These vectors are sparse: the query vector has only two non-zero components, one for “heart” and one for “disease”. An inverted file therefore reads just two posting lists, and a document that contains neither term is never touched. The embeddings of Building Semantic Search represent the same query as a dense vector with typically 768 components, all of them non-zero. Components can be positive or negative, so a document may gain score in one dimension and lose it in another, and even a small query component q_k matters when the matching document component $d_{i,k}$ is large. No document can be skipped because it lacks a query term. So far we have answered dense queries with a linear scan over all document vectors. Can we do better?

Text is not the only source of vectors. Later chapters on visual, audio, and video features describe images, sounds, and video shots by vectors of low-level perceptual features, such as the distribution of colors in an image. These features differ from embeddings in how they are obtained: a hand-designed extraction rule computes them directly from the signal. They also differ in how they are compared. Perceptual features usually come with a distance, such as Euclidean distance, where a smaller value means a better match. As with embeddings, the comparison function belongs to the feature. It follows from how the feature was defined or learned, and we cannot simply replace it with another one.

For both approaches, the search task is the same. A linear scan compares the query with every stored vector and returns the best matches. A better approach organizes the vectors in advance, as the inverted file does for sparse vectors, so that a query visits only a small part of the collection. This task is the **nearest-neighbor problem**. It is well understood in two and three dimensions, where geographic information systems and computer graphics have solved it for decades. With embeddings of hundreds or thousands of dimensions, it has gained new interest. This section formalizes what a best match is, first under a similarity score and then under a distance. The rest of the chapter asks how to find the best matches without a full scan.

Best-match search with similarity scores

Embeddings are compared with the dot product or with cosine similarity, the two measures of the vector space model in Vector Space Retrieval. Writing the query vector as $\boldsymbol{q}$ and a document vector as $\boldsymbol{d}_i$, the dot product sums the products of matching components, while cosine similarity divides that sum by the two vector lengths so that only direction matters.

Key Formula: Dot Product and Cosine Similarity

$$\mathrm{sim}_{\mathrm{dot}}(\boldsymbol{q}, \boldsymbol{d}_i) = \boldsymbol{q} \cdot \boldsymbol{d}_i = \sum_{k=1}^{M} q_k \, d_{i,k} \tag{6.1}$$

$$\mathrm{sim}_{\cos}(\boldsymbol{q}, \boldsymbol{d}_i) = \frac{\boldsymbol{q} \cdot \boldsymbol{d}_i}{\| \boldsymbol{q} \| \, \| \boldsymbol{d}_i \|} \tag{6.2}$$

The dot product rewards vectors that are both aligned and long; cosine similarity divides out the lengths and rewards alignment alone.

Figure 6.2 contrasts the geometry of the two measures. For cosine similarity (left), the best matches lie inside a cone around the query, and the length of a document vector plays no role. For the dot product (right), a hyperplane orthogonal to the query separates the best matches from the rest. Documents farther out along the query direction score higher, so the dot product favors long vectors. A document in the half-space opposite the query scores negatively under both measures.

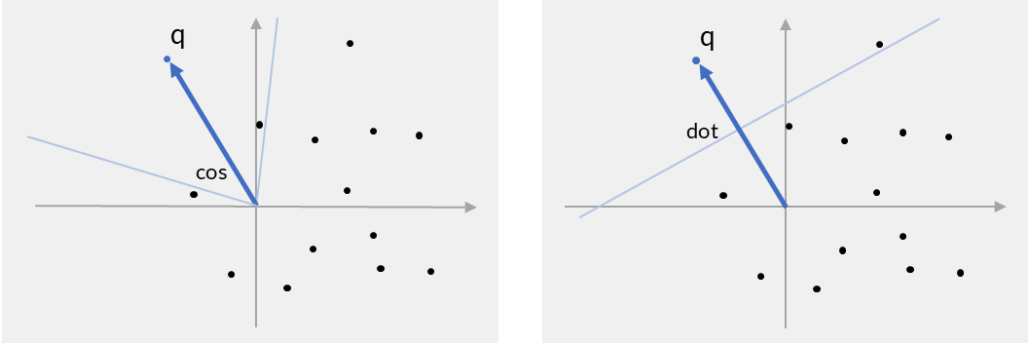


Figure 6.2: Cosine similarity (left) selects the documents inside a cone around $\boldsymbol{q}$; the dot product (right) selects the documents beyond a hyperplane orthogonal to $\boldsymbol{q}$.

With a comparison function in place, we can state the search problem. Given a query embedding $\boldsymbol{q}$ and a set $\mathbb{P}$ of stored document embeddings, we look for the documents that maximize the score.

Key Formula: Best-Match Search

$$\boldsymbol{d}^* = \arg\max_{\boldsymbol{d}_i \in \mathbb{P}} \mathrm{sim}(\boldsymbol{q}, \boldsymbol{d}_i), \qquad \mathrm{sim} \in \{\mathrm{sim}_{\mathrm{dot}}, \mathrm{sim}_{\cos}\} \tag{6.3}$$

More generally, the top-k result consists of the k documents with the highest scores. The definition says nothing about how to find them; a linear scan computes all scores and keeps the best.

In practice, the choice between the two measures is often made once, when the index is built. Most embedding models output vectors of unit length, and otherwise we normalize every document vector once before indexing, $\hat{\boldsymbol{d}}_i = \boldsymbol{d}_i / \parallel \boldsymbol{d}_i \parallel$. On the unit sphere, cosine similarity and dot product coincide, so the index only needs to compute dot products. The query needs no normalization at all:

$$\mathrm{sim}_{\cos}(\boldsymbol{q}, \boldsymbol{d}_i) = \frac{1}{\parallel \boldsymbol{q} \parallel} \boldsymbol{q} \cdot \hat{\boldsymbol{d}}_i \tag{6.4}$$

The factor $1/\parallel \boldsymbol{q} \parallel$ is the same for every document. It scales all scores of a query equally and leaves the ranking unchanged. Normalizing the query does no harm, and it is useful when scores must be comparable across queries, for example against a fixed similarity threshold. For ranking alone, it is not required.

Nearest-neighbor search with distances

To see how a perceptual feature turns media into a vector, consider the color content of an image. Figure 6.3 splits a photograph into its red, green, and blue channels and summarizes each channel by a histogram over its 256 intensity values. To build the histograms, we visit every pixel and increase three bins by one: the bin for the pixel's red intensity in the red histogram, and likewise for green and blue. The three histograms together hold $3 \times 256 = 768$ counts, which we read as a 768-dimensional feature vector of the image.

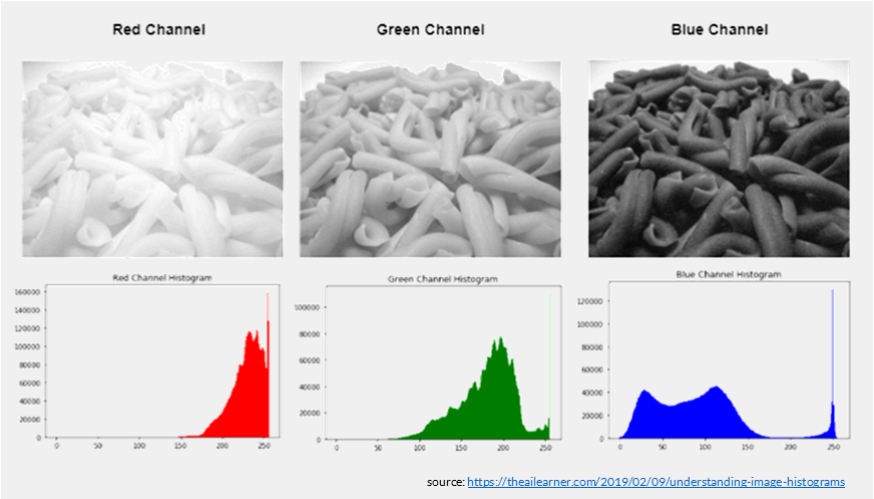


Figure 6.3: A color image split into its red, green, and blue channels, each summarized by an intensity histogram.

Exact intensities are too fine-grained for comparing images. Two photographs of the same pasta under slightly different light would share hardly any bins. We therefore quantize each histogram into fewer, wider bins, for example 16 bins per channel that each cover 16 intensity values. Similar shades now fall into the same bin, and the feature vector shrinks to $3 \times 16 = 48$ values, each counting the pixels in one bin.

To compare two images by their colors, we sum the absolute differences of their counts over all 48 bins. Two images with similar color distributions differ little in every bin and receive a small sum. A photograph of a blue sky differs strongly from the red pasta in many bins and receives a large sum. This sum is the Manhattan distance between the two feature vectors. The color histogram is deliberately simple: it only shows how image content is reduced to a vector. Later chapters introduce more expressive perceptual features.

A **distance** $\delta(\boldsymbol{q}, \boldsymbol{p}_i)$ between the query $\boldsymbol{q}$ and a stored vector $\boldsymbol{p}_i$ reverses the direction of comparison: a smaller value means a better match, and identical vectors have distance zero. Most distances used in retrieval are metrics, which means they are non-negative, symmetric, and satisfy the triangle inequality. Each distance induces a different notion of a neighborhood, that is, a different shape for the set of points within a fixed distance of a center. Figure 6.4 shows these shapes in 2D.

- **Euclidean distance** (L2) measures straight-line distance and gives a round neighborhood. It is the default for continuous features.
- **Manhattan distance** (L1) sums the absolute differences per dimension and gives a diamond neighborhood. It emphasizes many small differences.
- **Maximum distance** (L-infinity) takes the single largest component difference and gives a square neighborhood. It lets the worst dimension dominate.
- **Quadratic distance** weights and correlates the dimensions through a matrix, giving a tilted-ellipse neighborhood.

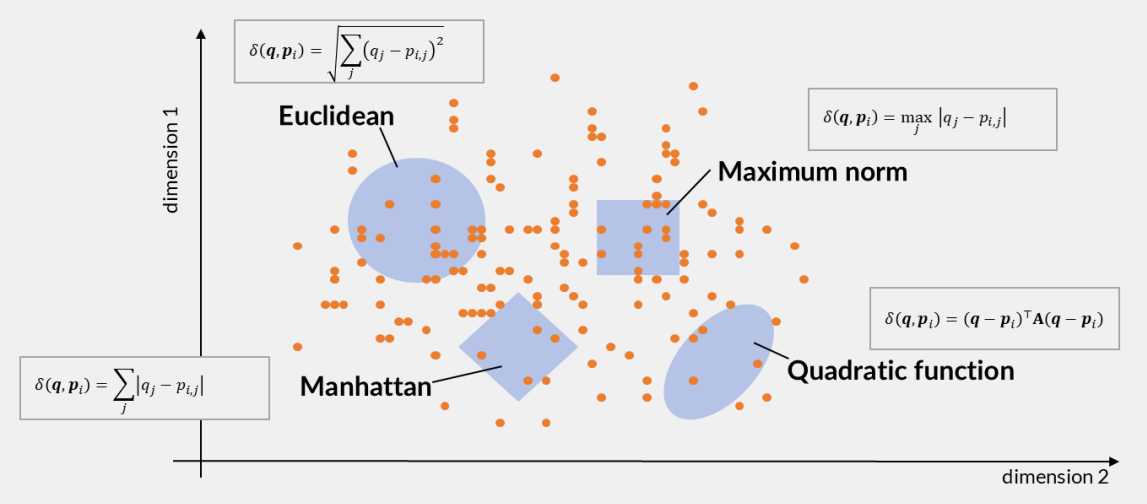


Figure 6.4: Neighborhood shapes of four distance measures in 2D: Euclidean (round), Manhattan (diamond), maximum (square), and quadratic (tilted ellipse).

Distances and similarity scores are two views of the same comparison. On the unit sphere the link is exact. For normalized vectors, the squared Euclidean distance is a linear function of the dot product:

$$\| \hat{\boldsymbol{q}} - \hat{\boldsymbol{d}}_i \|^2 = 2 - 2\, \hat{\boldsymbol{q}} \cdot \hat{\boldsymbol{d}}_i \tag{6.5}$$

Ranking normalized vectors by increasing Euclidean distance therefore returns the same order as ranking them by decreasing cosine similarity. In general, any distance can be turned into a similarity score with a decreasing function, such as $1/(1+\delta)$, without changing the ranking. Such a conversion keeps the comparison and changes only the scale of the values. Replacing the comparison is a different matter. Using cosine similarity instead of the Manhattan distance on color histograms changes the ranking, and only the design of the feature can justify that change.

One awkward property of distances is that they are hard to read on their own. A cosine of 0.9 clearly signals a strong match, but a distance of 32.8 says nothing until it is compared with other distances. This is one reason why normalized similarity scores are convenient when a human or a downstream system needs to interpret the score.

With a distance, the search problem keeps its form. We minimize instead of maximize, and the best match becomes the nearest neighbor.

Key Formula: Nearest-Neighbor Search

$$\mathrm{NN}(\boldsymbol{q}) = \arg\min_{\boldsymbol{p}_i \in \mathbb{P}} \delta(\boldsymbol{q}, \boldsymbol{p}_i) \tag{6.6}$$

The k nearest neighbors are the k stored vectors with the smallest distances. This is best-match search with the direction of comparison reversed; for normalized vectors under Euclidean distance, both return the same result.

Keeping dimensions comparable

Embeddings rarely need further adjustment, because every dimension comes from the same learned space and lives on a comparable scale. Perceptual features are different. A feature vector often combines values that were measured independently, and the color histogram shows two forms of the resulting problem.

The first is scale. Bin counts depend on the size of the image: a photograph with four times as many pixels has four times larger counts, so the Manhattan distance between a small and a large image of the same scene is large. This is the same problem the dot product has with long documents in Vector Space Retrieval, and the fix is similar. We divide every count by the number of pixels. Each channel's histogram then holds fractions that sum to 1, independent of the image size.

The second is mixing features. Suppose we append the image's average brightness, measured on a scale from 0 to 255, to the 48 normalized bins. The bins of each channel sum to 1, so the Manhattan distance over all 48 bins can never exceed 6. A brightness difference of 50 alone outweighs any possible color difference, and the distance effectively compares brightness only. To give each dimension a fair share, we rescale it by its spread over the collection. For each dimension j, we subtract the mean μ_j and divide by the standard deviation σ_j of that dimension, both estimated once from the collection or from a representative sample.

$$\hat{v}_j = \frac{v_j - \mu_j}{\sigma_j} \tag{6.7}$$

After this **Gaussian normalization**, every dimension has mean 0 and unit variance. A difference of one unit then means the same in every dimension: one standard deviation of the typical variation. Equivalently, the distance weights each dimension's difference by $1/\sigma_j$, so each dimension contributes in proportion to how much it actually varies.

Normalize on statistics, not on a single query

Gaussian normalization uses the mean and standard deviation of the whole collection, not of one query or one result set. Estimate μ_j and σ_j once from a representative sample and keep them fixed, so that the same vector always maps to the same normalized point regardless of which query retrieves it.

Gaussian normalization treats the dimensions as independent, but the bins of a color histogram are not. The bin for red is related to the bin for orange, because the two colors are perceptually close, while red and blue are not. The **quadratic distance** captures such relationships through a positive semi-definite matrix $\boldsymbol{A}$, whose eigenvectors define the main axes of the data and whose eigenvalues scale them.

Computing $(\boldsymbol{q} - \boldsymbol{p}_i)^\top \boldsymbol{A} (\boldsymbol{q} - \boldsymbol{p}_i)$ for every candidate is expensive, but an eigenvalue decomposition avoids it. Rotating and scaling the space along the eigenvectors of $\boldsymbol{A}$, shown in Figure 6.5, maps the quadratic distance in the original space onto a plain Euclidean distance in the transformed space. We transform all

vectors once when building the index and then compare them with ordinary Euclidean distance at query time.

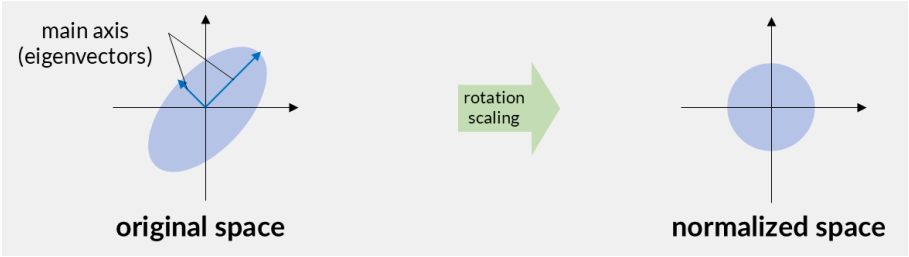


Figure 6.5: Whitening: rotating along the eigenvectors of the weighting matrix and scaling by the square roots of its eigenvalues turns a correlated distribution into an isotropic one.

Whether we maximize a similarity score or minimize a distance, we now have a precise definition of the best matches. The rest of the chapter answers a single question about these definitions: how do we compute the arg-max or arg-min without comparing the query to every stored vector? For a few thousand vectors a full scan is fine. For millions or billions it is not, and the structures that speed up the search behave very differently depending on how many dimensions the vectors have.

Hands-on: Distance measures and normalization

Compute dot product, cosine, and Euclidean and Manhattan distances on the same vectors, watch length normalization make cosine and dot product agree, and see Gaussian normalization rebalance dimensions on different scales. Open notebook ->

Includes pre-run results: you can read through or download and experiment.

Low-Dimensional Search Structures

A city has a few dozen post offices, and a resident wants to know which one is closest to home. This is the nearest-neighbor problem of the previous section in two dimensions, and Donald Knuth used exactly this scenario to name it: the post-office problem. In two dimensions the problem is easy, and its solution has been known for a long time. This section follows how the solutions for two and three dimensions grew from a precomputed map of all answers into tree structures, and why the hope of searching in logarithmic time, as a B-tree does for one-dimensional keys, was never quite fulfilled.

Precomputing all answers: Voronoi diagrams

For the post-office problem, we can draw a map that answers every possible query in advance. Around each post office, we mark the region of all locations that are closer to this office than to any other. The result is a **Voronoi diagram**: a partition of the space into one cell per stored point, as shown in Figure 6.6. To answer a query, we locate the cell that contains the resident's home and return the post office of that cell.

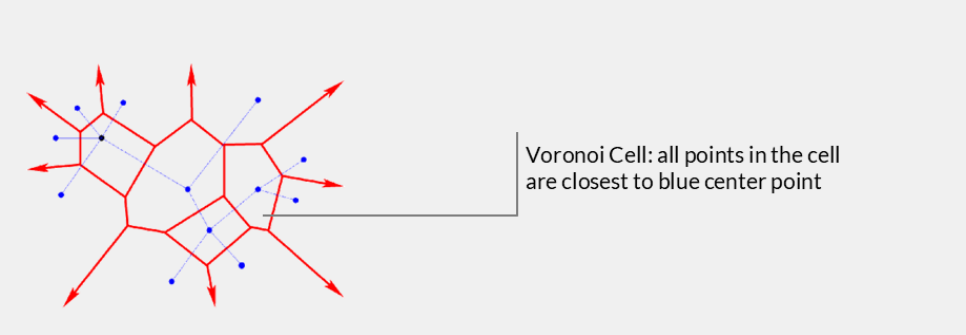


Figure 6.6: A Voronoi diagram: every location inside a cell is closer to the cell's seed point than to any other seed point.

The Voronoi diagram inverts the search problem. Instead of searching for the nearest neighbor of one query, it computes the nearest neighbor of all possible queries at once and stores the answers as regions. This is the geometric analogue of the inverted file. An inverted file precomputes the result of every single-term query; a Voronoi diagram precomputes the result of every nearest-neighbor query. The idea is not tied to Euclidean distance: Voronoi diagrams exist for other distance measures as well.

The difficulty lies in building and storing the cells. In two dimensions, the diagram of n points has a size proportional to n. In d dimensions, its worst-case size grows like $n^{\lceil d/2 \rceil}$, so already in three dimensions it can grow quadratically, and finding the cell that contains a query becomes as hard as the search it was meant to replace. A common workaround approximates each cell by its minimum bounding rectangle, but the rectangles of neighboring cells overlap, so a query must check several candidates instead of one. Insertions and deletions are awkward too, since one new point reshapes all neighboring cells. Voronoi diagrams remain the clearest mental model of exact nearest-neighbor search, but not a practical index beyond a few dimensions.

From B-trees to spatial indexes

Spatial search is much older than embeddings. Geographic information systems, computer-aided design, and image databases needed it from the 1970s onward, and the research followed a familiar path. It started with simple partitions of space and moved on to tree structures, in the hope of repeating for two and three dimensions what the B-tree achieves for one-dimensional keys: finding any entry in a logarithmic number of steps.

These structures were designed for **range search** first. A range query asks for all points within a region, for example all points within distance r of the query $\boldsymbol{q}$, as in Figure 6.7, or all points inside a map window. The region is known before the search starts, so the index only needs to identify the parts of the space that overlap it.

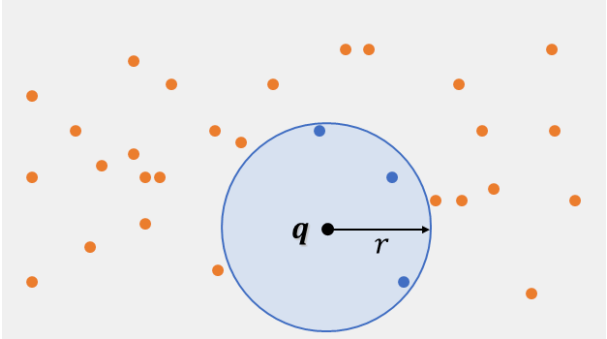


Figure 6.7: A range query returns all points within radius r of the query $\boldsymbol{q}$.

The **grid file** is the simplest such structure. It lays a grid over the space, chooses the grid lines so that each cell holds roughly the same number of points, and stores the points of each cell on a disk page, as shown in Figure 6.8. A directory maps every cell to its page. A range query reads only the pages of the cells that overlap the query region. In two dimensions, a small query window needs only a few page reads, however large the collection is.

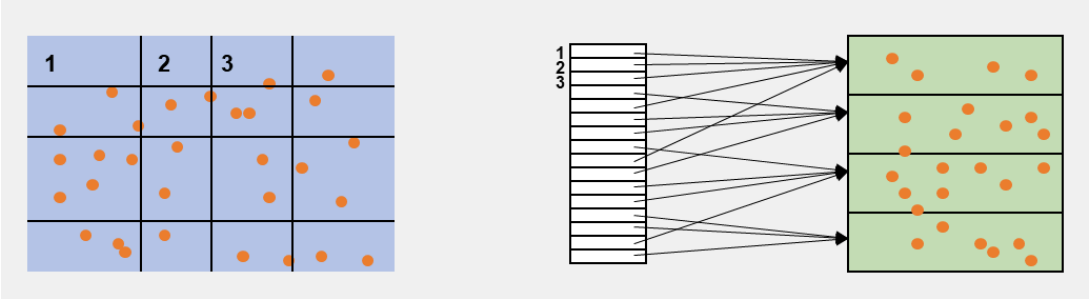


Figure 6.8: A grid file partitions the space into cells (left) and maps each cell through a directory to a disk page holding its points (right).

How a grid file grows (optional reading)

The grid lines of a grid file are not equally spaced. They are placed so that the resulting disk pages hold roughly the same number of points, which keeps the cost of reading a page predictable.

The directory associates each grid cell with the disk page that holds its points. Many cells stay empty or nearly empty when the data is unevenly distributed, so several cells may share one disk page. This keeps the number of pages close to the minimum needed for the data.

When an insertion fills a page that serves several cells, the grid file creates a new page and distributes the cells evenly between the two pages. Only the directory entries of these cells change. When a single cell grows too large for one page, the grid file adds a new partition line along a chosen dimension. This splits every cell that the line crosses and enlarges the directory, but again only the points of the overflowing cell move to new pages.

The directory is also the grid file's weakness. Splitting each of d dimensions only once already produces 2^d cells. For the 48-dimensional color histograms of the previous section this is $2^{48} \approx 2.8 \times 10^{14}$ cells, almost all of them empty, and the directory no longer fits in memory.

R-trees and the problem of overlap

The **R-tree** carries the idea of the B-tree directly into the multi-dimensional space. It groups nearby points into a **minimum bounding rectangle** (MBR), groups nearby rectangles into larger rectangles, and continues up to a single root. Figure 6.9 shows a small R-tree. Like a B-tree, it is balanced: all leaves are at the same depth, and every node holds between a minimum and a maximum number of entries. When a node overflows, it is split into two nodes, and the split may propagate up to the root. R-trees index points, rectangles, and polygons, and they are a mainstay of spatial databases to this day.

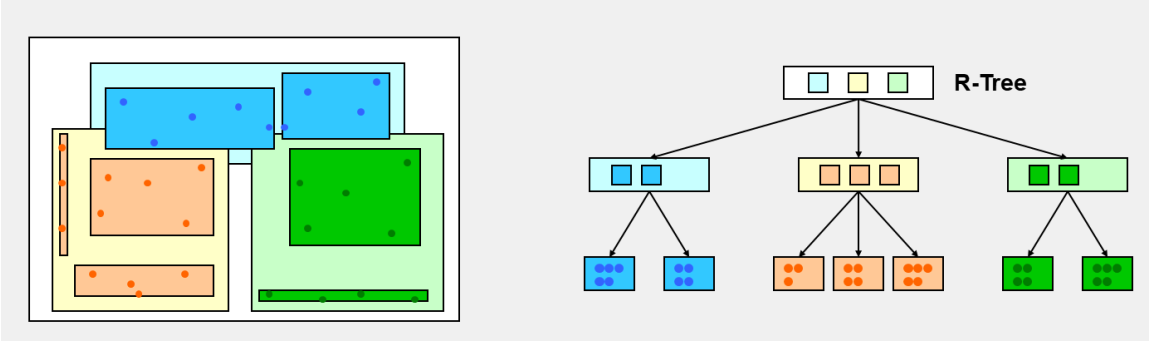


Figure 6.9: An R-tree groups points into nested minimum bounding rectangles (left) that form a balanced tree (right).

Splitting a leaf is easy. We divide its points at the median of one dimension, and the two new rectangles do not overlap. Splitting an inner node is harder, because its entries are rectangles, not points, and two groups of rectangles often cannot be separated cleanly. Figure 6.10 shows the possible outcomes: a clean split, a split whose rectangles overlap, and a split where one rectangle lies completely inside the other.

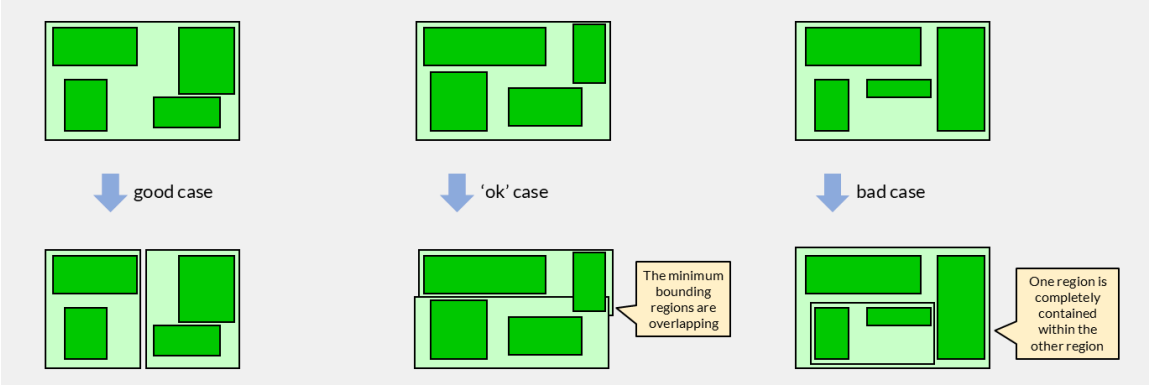


Figure 6.10: Splitting an inner node: a clean split (left), overlapping rectangles (middle), and one rectangle contained in the other (right).

Overlap also grows over time. When a new point falls into the overlap of two rectangles, as in Figure 6.11, the insertion must choose a path. The usual heuristic follows the child whose rectangle needs the least enlargement. A poor choice inflates a rectangle, which then overlaps its neighbors even more and makes the next decision harder. Every query that reaches an overlapping region must follow several branches, and the tree loses the single root-to-leaf path that makes a B-tree fast.

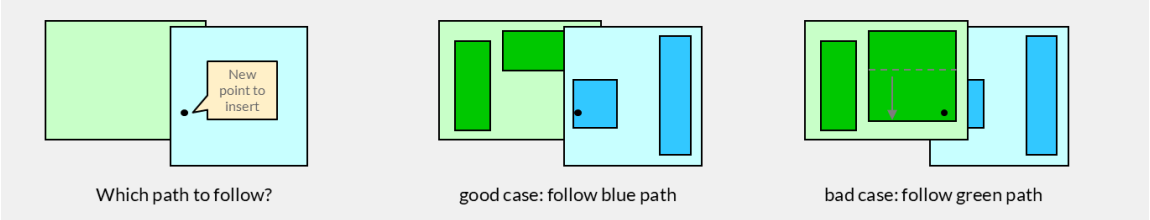


Figure 6.11: Inserting a point into overlapping rectangles: the blue path keeps the rectangles tight, the green path enlarges them.

A second problem is less visible but more fundamental: nearest-neighbor search is not a range search. A range query knows its region in advance. A nearest-neighbor query does not know how far away its answer lies. It must find a candidate and then prove that no other point is closer. If the query lies near the border of a leaf, the closest point may well sit in a neighboring leaf, possibly in a different subtree, and the search has to look there before it can stop. Even without any overlap, a nearest-neighbor search therefore tends to visit more than one path from the root to the leaves.

An optimal nearest-neighbor algorithm

Hjaltason and Samet proposed a nearest-neighbor algorithm that works for any structure that divides the space into nested regions: grid files, R-trees, and their many variants. It is optimal in a precise sense. It

finds the nearest neighbor and proves that no other point is closer, and no correct algorithm can do this with fewer node inspections.

The algorithm keeps a priority queue of nodes and points, ordered by their distance to the query $\boldsymbol{q}$. For a point, this is the exact distance. For a node, it is a lower bound: the smallest distance that any point inside the node's region could have.

```
insert the root into the queue with its lower bound
while the queue is not empty:
    remove the entry e with the smallest distance
    if e is a data point:   return e as the nearest neighbor
    if e is a leaf:         insert every point of e with its distance to q
    if e is an inner node:  insert every child of e with its lower bound
```

The algorithm only needs two primitives: the distance between the query and a point, and a lower bound for the distance between the query and a region. For a rectangle and Euclidean distance, the lower bound is the distance to the closest point of the rectangle.

Key Formula: Lower Bound to a Bounding Rectangle

For a rectangle with per-dimension bounds $[l_j, u_j]$, the smallest possible L2 distance from the query $\boldsymbol{q}$ is

$$\mathrm{lBnd}(\boldsymbol{q}, \mathrm{MBR}) = \sqrt{\sum_j} \tag{6.8}$$

Dimensions in which the query lies inside the rectangle contribute nothing; the others contribute the gap to the nearer edge.

To find the k nearest neighbors, the algorithm continues until k data points have been removed from the queue. The following example shows what the search costs even in two dimensions.

Example

An R-tree stores twelve points in four leaves. The inner node A holds the leaves L1 and L2, and the inner node B holds L3 and L4. The query is $\boldsymbol{q} = (4.6, 4.6)$, which lies in the gap between the two subtrees. For leaf L1 with rectangle $[1, 3.8] \times [1, 3.4]$, the query lies to the right of and above the rectangle, so $\mathrm{lBnd} = \sqrt{(4.6-3.8)^2 + (4.6-3.4)^2} = \sqrt{2.08} = 1.44$. For L3 with rectangle $[5.2, 8] \times [5.4, 9]$, it is $\sqrt{0.6^2 + 0.8^2} = 1.00$.

Node	Contains	Rectangle	lBnd
A	L1, L2	$[1, 3.8] \times [1, 8]$	0.80
B	L3, L4	$[5.2, 8] \times [1, 9]$	0.60
L1	$(1, 1)$, $(3, 1.5)$, $(3.8, 3.4)$	$[1, 3.8] \times [1, 3.4]$	1.44
L2	$(1, 6)$, $(2.5, 8)$, $(3, 5.6)$	$[1, 3] \times [5.6, 8]$	1.89
L3	$(5.2, 7)$, $(6.5, 5.4)$, $(8, 9)$	$[5.2, 8] \times [5.4, 9]$	1.00
L4	$(6.4, 1)$, $(8, 2.5)$, $(7, 3.8)$	$[6.4, 8] \times [1, 3.8]$	1.97

The search proceeds as follows:

Step	Removed from the queue	Inserted into the queue
1	root	B (0.60), A (0.80)
2	B (0.60)	L3 (1.00), L4 (1.97)
3	A (0.80)	L1 (1.44), L2 (1.89)

4	L3 (1.00)	$(6.5, 5.4)$ at 2.06, $(5.2, 7)$ at 2.47, $(8, 9)$ at 5.56
5	L1 (1.44)	$(3.8, 3.4)$ at 1.44, $(3, 1.5)$ at 3.49, $(1, 1)$ at 5.09
6	$(3.8, 3.4)$ at 1.44	none: nearest neighbor found

The leaf L3 looked most promising, but its closest point is 2.06 away. The nearest neighbor $(3.8, 3.4)$ lies in the other subtree. The search followed two paths from the root to the leaves, root-B-L3 and root-A-L1, and opened two of the four leaves. L2 and L4 were never opened, because their lower bounds exceed the distance 1.44 of the nearest neighbor.

The example shows the cost structure of every tree-based nearest-neighbor search. The search visited both inner nodes and two leaves, twice as many as a single root-to-leaf path. Because the algorithm is optimal, nothing can be done about this: any correct algorithm must open L3, since a point within distance 1.00 of the query could have been there. The extra paths are the price of proving the answer, not a weakness of the algorithm.

Why the algorithm is correct and optimal (optional reading)

Correctness. The queue orders its entries by increasing distance. Every child of a node lies inside the node's region, so its lower bound is at least as large as the node's lower bound, and every point inside a region is at least as far away as the region's lower bound. When a data point reaches the front of the queue, every entry still in the queue, and every descendant of these entries, is at least as far from the query as this point. No closer point can remain hidden, so the point at the front is the nearest neighbor.

Optimality. Suppose we already knew the nearest neighbor of q. The **NN-sphere** is the sphere centered at q that passes through this nearest neighbor, drawn in Figure 6.12. Any node whose region intersects the NN-sphere might contain a closer point, so any correct algorithm must inspect it; skipping it would leave the answer unproven. A node whose region lies entirely outside the NN-sphere cannot contain a closer point and need not be inspected.

The algorithm inspects nodes in increasing order of their lower bounds and stops as soon as the nearest neighbor reaches the front of the queue. At that moment, it has inspected exactly the nodes whose lower bound is at most the nearest-neighbor distance, which are the nodes whose regions intersect the NN-sphere. It therefore inspects no node that a correct algorithm could skip. In the example above, the NN-sphere has radius 1.44 and intersects A, B, L1, and L3, which are exactly the nodes the search opened.

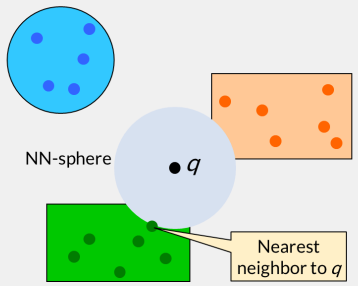


Figure 6.12: The NN-sphere around q: regions that intersect it must be inspected, regions outside it can be skipped.

The same argument extends to the k nearest neighbors, with the sphere passing through the k-th nearest neighbor.

Beyond Euclidean distance: dot product and cosine

The algorithm never looks inside the distance function. It needs a bound for each region that is optimistic, meaning it never promises less than the region could deliver. For a distance, this is a lower bound; for a similarity score, it is an upper bound, and the queue is ordered by decreasing score instead of increasing distance. Figure 6.13 shows both cases.

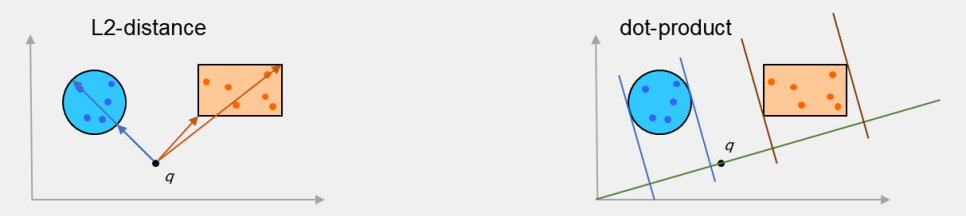


Figure 6.13: Bounding a region: for L2 distance (left), the nearest and farthest points of the region bound the distance; for the dot product (right), two hyperplanes orthogonal to $\boldsymbol{q}$ bound the similarity.

For the dot product, the bounds follow from the fact that the score is a sum of component-wise products. Each product $q_j p_j$ is largest at the upper edge u_j if q_j is positive and at the lower edge l_j if q_j is negative:

$$\mathrm{uBnd}(\boldsymbol{q}, \mathrm{MBR}) = \sum_j \tag{6.9}$$

For a spherical region with center $\boldsymbol{c}$ and radius r, the bound is $\boldsymbol{q} \cdot \boldsymbol{c} + r \parallel \boldsymbol{q} \parallel$, attained by the point of the sphere that lies farthest in the direction of $\boldsymbol{q}$. Ram and Gray used this bound to run exact maximum-inner-product search on ball trees.

So do these structures work with dot product and cosine out of the box? Largely yes, with a few caveats.

- **Grid files and R-trees** work with the dot product as soon as the bound function is replaced. Nested rectangles keep the property the algorithm relies on: a child's bound never exceeds its parent's bound. Correctness is therefore guaranteed. Efficiency is not, because the split and insertion heuristics were designed to keep rectangles small under Euclidean distance, not to group vectors that score similarly.
- **Cosine similarity** needs no special treatment. After normalizing the stored vectors, ranking by cosine equals ranking by Euclidean distance, as the previous section showed. Any Euclidean index can be used unchanged.
- **Metric trees** such as the M-tree prune with the triangle inequality. The dot product is not a metric, so these structures cannot use it directly. With cosine, they can again work on the Euclidean distance between normalized vectors.
- **Voronoi diagrams** change their character. Under the dot product, a point is not necessarily its own best match: for $\boldsymbol{p} = (1, 1)$, the longer vector $(2, 2)$ scores 4 against $\boldsymbol{p}$, while $\boldsymbol{p}$ scores only 2 against itself. Only the vertices of the convex hull of the collection can be the best match of any query; all other points have empty cells.

There is also a general trick that turns maximum-inner-product search into nearest-neighbor search. Let M be the largest vector length in the collection. Appending the component $\sqrt{M^2 - \parallel \boldsymbol{p}_i \parallel^2}$ to every stored vector and 0 to the query gives all stored vectors the same length M. The squared Euclidean distance between the extended vectors is then $\parallel \boldsymbol{q} \parallel^2 + M^2 - 2\,\boldsymbol{q} \cdot \boldsymbol{p}_i$, so the nearest neighbor is exactly the vector with the largest dot product, and any Euclidean index can answer the query.

A fundamental limit

In two and three dimensions, the extra paths are a small price. A nearest-neighbor search opens a few more leaves than a range search would, and the structures easily beat a full scan. Research in the 1990s, however, showed that the price grows with the number of dimensions. Rectangles overlap more, the nearest neighbor lies farther away, and the NN-sphere intersects more and more leaves, until the optimal algorithm visits almost all of them.

The research community responded with a long series of new structures. They changed the shape of the bounding regions to spheres or combinations of spheres and rectangles, introduced split strategies that reduce overlap, enlarged nodes when splitting did not pay off, or replaced the data-space metric by an object

metric such as edit distance. Notable examples include the R+-tree (1987), R*-tree (1990), TV-tree (1994), vp-tree (1993), GiST (1995), X-tree (1996), SS-tree (1996), SR-tree (1997), M-tree (1997), and Pyramid-tree (1998), and many more followed. Each improved on its predecessors for some data and some dimensions. None of them restored logarithmic search in high dimensions.

That so many clever designs failed in the same way suggests that the problem lies not in any single structure but in high-dimensional space itself. The effect is known as the **curse of dimensionality**, a term originally coined by Richard Bellman. The next section explains what makes high-dimensional spaces so hostile to search.

The Curse of Dimensionality

Optional Background Reading

This section explains why exact nearest-neighbor search fails in high dimensions, with the geometry and statistics behind it, and it is here as background. The next section only needs its conclusions, so a reader focused on the search methods can skim or skip it. The next section opens with a short recap of what carries forward.

All the structures of the previous section were developed with two or three dimensions in mind. Their inventors argued with drawings in the plane: rectangles that enclose points, spheres that touch rectangles, regions that can be skipped. When feature vectors with dozens or hundreds of dimensions arrived, the natural expectation was that these arguments would carry over, and that an index would prune the search in 100 dimensions much as it does in 2. This intuition turned out to be wrong. This section shows why, first for distances and then for the dot product, and draws a conclusion that shapes the rest of the chapter: in high dimensions, the exact nearest neighbor is not worth chasing.

Distances grow with the dimension

Take the unit square and two points that are clearly close: the center $\boldsymbol{c} = (0.5, 0.5)$ and a point $\boldsymbol{p} = (0.4, 0.4)$ slightly off-center. Around $\boldsymbol{p}$ we draw a circle of radius $r = 3$, shown in Figure 6.14. The farthest corner of the square is only 0.85 away from $\boldsymbol{p}$, so the circle covers the whole square many times over. A radius of 3 looks absurdly large.

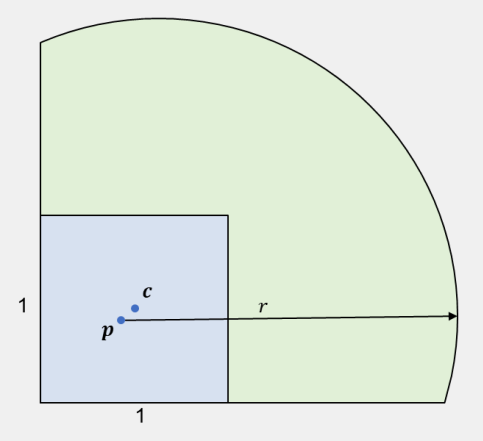


Figure 6.14: A circle of radius r around $\boldsymbol{p}$, close to the center $\boldsymbol{c}$ of the unit square.

Now we add dimensions, keeping the same pattern: $\boldsymbol{c}$ has the value 0.5 and $\boldsymbol{p}$ the value 0.4 in every dimension. The two points differ by 0.1 in each of d dimensions, so their Euclidean distance is $0.1\sqrt{d}$. The farthest corner of the unit cube lies at distance $0.6\sqrt{d}$ from $\boldsymbol{p}$. Both distances grow with $\sqrt{d}$, while the radius stays fixed:

- Up to $d = 25$, the sphere of radius 3 still contains the whole unit cube, because $0.6\sqrt{25} = 3$.
- Beyond $d = 25$, corners of the cube start to fall outside the sphere, and their number grows quickly.
- Beyond $d = 900$, even the center $\boldsymbol{c}$ falls outside, because $0.1\sqrt{900} = 3$.

Nine hundred dimensions are common for features extracted from media. In such a space, a point we would call “close” by any 2D standard lies outside a radius that covers the entire square in 2D. Distances simply grow with the dimension, and they grow for all pairs of points alike.

Distances lose their contrast

Growing distances alone would only change the scale. The real problem is that the distances of all points to a query grow at the same pace, so they become ever more similar. A simple measure for this is the **contrast** between the farthest and the nearest point.

Key Formula: Loss of Contrast

For a query and a fixed set of data points, let $\delta_{\max}$ and $\delta_{\min}$ be the distances to the farthest and the nearest point. For independent random data, as $d \to \infty$:

$$\frac{\delta_{\max} - \delta_{\min}}{\delta_{\min}} \to 0 \tag{6.10}$$

The farthest point is barely farther away than the nearest one; "nearest" loses its distinction.

Example

We place 10,000 random points uniformly in the unit cube and measure the contrast from random queries, averaged over 50 queries:

Dimensions d	Contrast, Manhattan (L1)	Contrast, Euclidean (L2)	Mean L2 distance	Std. dev. of L2 distance
2	401	355	0.53	0.23
10	5.2	4.2	1.28	0.23
100	0.69	0.54	4.10	0.23
1000	0.17	0.14	12.90	0.22

In 2D, the farthest point is hundreds of times farther away than the nearest one. At $d = 1000$, it is only 17 percent (L1) or 14 percent (L2) farther. The last two columns show why: the mean distance grows like $\sqrt{d}$, but the spread around it stays at about 0.23. The same spread on top of an ever larger distance means ever less contrast. The Manhattan distance keeps a little more contrast than the Euclidean distance, but both collapse.

Figure 6.15 shows the same effect as distributions. In two dimensions, distances spread over a wide range; at $d = 8192$, they pile up around a mean of about 36.9 with a standard deviation near 0.24, so that 99 percent of all pairs fall into a band barely more than one unit wide.

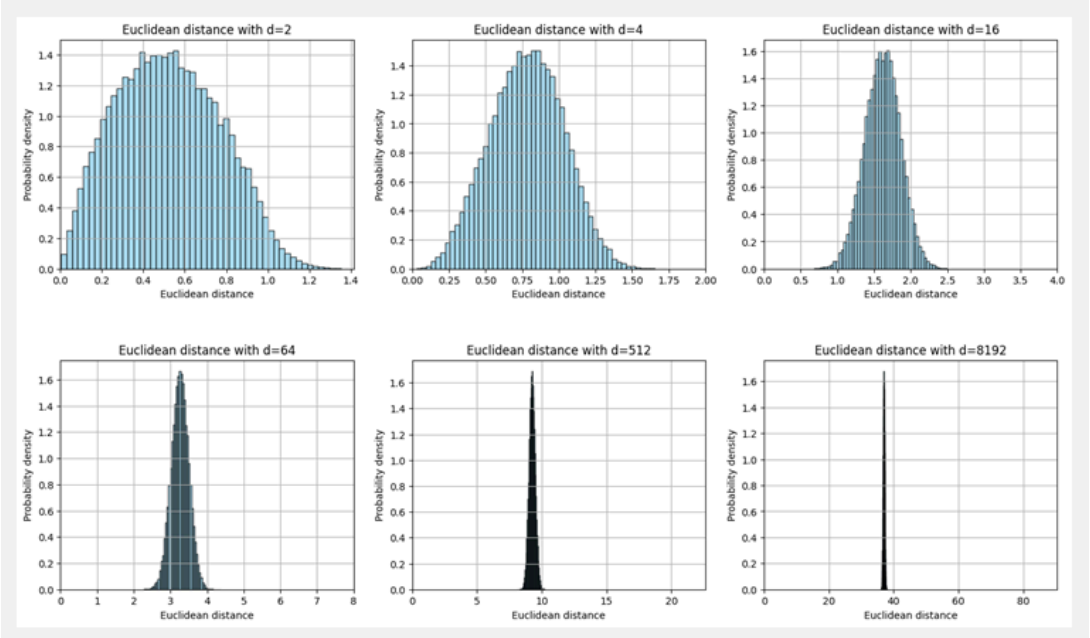


Figure 6.15: Distributions of Euclidean distances between random points in the unit cube, from $d = 2$ to $d = 8192$.

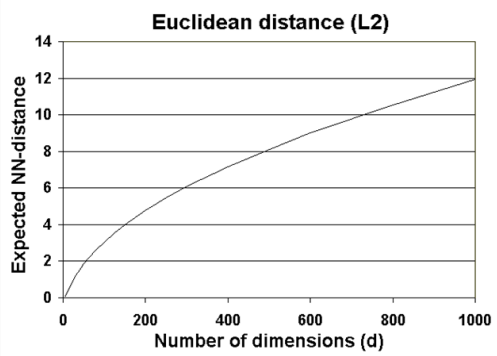
The loss of contrast has two consequences. The first concerns the index structures of the previous section; the second concerns the meaning of the nearest neighbor itself.

Consequence 1: trees can no longer prune

The optimal algorithm of the previous section must inspect every node whose region intersects the NN-sphere, the sphere around the query that passes through the nearest neighbor. If the nearest neighbor is far away, this sphere is large, and few regions can be skipped. To estimate the cost, we use a simple model: the data space is the unit cube, the dimensions are independent and uniformly distributed, and we use Euclidean distance. In this model, the probability that a point falls into a region equals the region's volume.

Two quantities compete, shown side by side in Figure 6.16. The first is the **expected nearest-neighbor distance**, which grows with the dimension (left). For one million points, it is 0.24 at $d = 10$, 1.37 at $d = 40$, 2.03 at $d = 60$, and 2.97 at $d = 100$.

The second is the size of a leaf (right). A tree cannot split every dimension. Each split in the middle of a dimension halves the number of points in a region, so one million points support only about 14 splits: after that, a leaf holds $10^6/2^{14} \approx 61$ points, about one disk page. A leaf is therefore split along $d' \approx 14$ dimensions and spans the full range in all others. The largest distance from any query to such a leaf is $l_{\max} = 0.5\sqrt{d'} \approx 1.87$, no matter how many dimensions the space has.



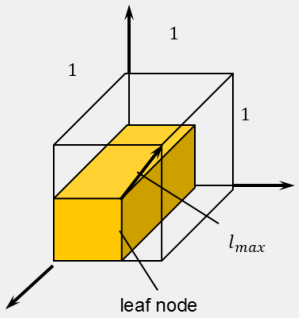


Figure 6.16: Left: the expected nearest-neighbor distance grows roughly like $\sqrt{d}$. Right: a leaf split along d' dimensions; no query in the unit cube is farther than $l_{\max} = 0.5\sqrt{d'}$ from it.

Once the expected nearest-neighbor distance exceeds $l_{\max}$, every leaf lies closer to the query than the query's own nearest neighbor. Every leaf intersects the NN-sphere, and even the optimal algorithm must visit all of them. For one million points, this happens between $d = 40$ and $d = 60$. Beyond that point, the hierarchy only adds overhead: why build a tree if every leaf must be read anyway? In practice the index loses even earlier. A full scan reads the data sequentially, while a tree reads its leaves one by one; once a tree must visit a sizeable fraction of its leaves, the scan is faster. Weber, Schek, and Blott showed that for uniform data this happens beyond roughly ten dimensions (see the Further Reading section of the chapter summary).

Two more surprises of high-dimensional space (optional reading)

The growth of distances is one of several ways in which high-dimensional space defies intuition. Two further puzzles make the point. Each one ends with a question: try to answer it before you open the resolution.

The escaping sphere. Place a square of side 2 with a circle of radius 1 at each corner, so that the circles cover all edges of the square. A further circle at the center just touches the four corner circles, as in Figure 6.17.

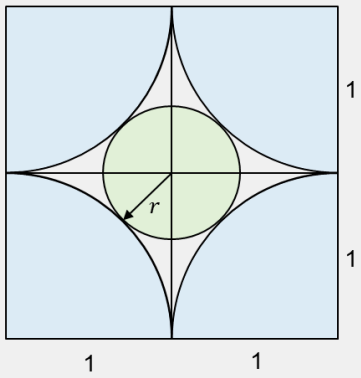


Figure 6.17: The escaping-sphere puzzle in 2D: four corner circles of radius 1 and a central circle that touches all of them.

What is the radius r of the central circle? Now repeat the construction in d dimensions, with a hypersphere at each corner of the hypercube and a central hypersphere that touches all of them. Does the central sphere stay inside the cube?

Resolution

The distance from the center of the square to a corner is $\sqrt{2}$, so the central circle has radius $r = \sqrt{2} - 1 \approx 0.41$. In d dimensions, the distance from the center of the hypercube to a corner is $\sqrt{d}$, so $r = \sqrt{d} - 1$. At $d = 4$ the radius is exactly 1, and the central sphere touches the faces of the cube. At $d = 100$ it is 9, and the "inner" sphere reaches far outside the cube that supposedly contains it.

The reason is that the corner spheres cover less and less of each face as the dimension grows. Already in 3D, the center of each face is left uncovered, and in higher dimensions the central sphere escapes through these gaps.

Where have all the points gone? Place a smaller square of side $s = 0.99$ at the center of the unit square, as in Figure 6.18, and distribute points uniformly over the unit square. The inner square seems to cover almost everything.

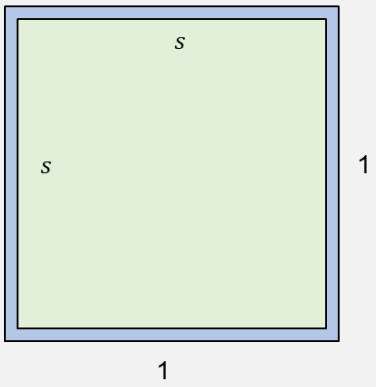


Figure 6.18: An inner square of side s centered in the unit square.

What fraction of the points lies inside the inner square? How does this fraction change in d dimensions, and where are the points that are not inside?

Resolution

The fraction equals the volume of the inner square. In 2D it is $0.99^2 \approx 98$ percent. In d dimensions it is 0.99^d: about 37 percent at $d = 100$ and only 4.3×10^{-5} at $d = 1000$. Of one million uniformly distributed points, only about 43 lie inside the inner cube.

> In each single dimension, a point lies inside the inner range with probability 0.99. But with 1000 dimensions, the probability that it lies outside in at least one of them is $1 - 0.99^{1000} = 0.99996$. Almost every point lies within 0.005 of one of the $2d$ faces of the cube. The center of a high-dimensional space is empty, and any partition of the space into regions produces regions that almost certainly contain no data.

Consequence 2: what does "nearest" still mean?

The second consequence is more fundamental. If all points are almost equally far from the query, the nearest neighbor is only marginally nearer than the rest. Proximity stops expressing similarity: a point that is 14 percent farther away than the nearest neighbor is not meaningfully less similar.

Real data is kinder than this model. The analysis above assumes uniformly distributed, independent dimensions, and real feature vectors and embeddings are neither. They form clusters: images of sunsets lie closer to each other than to images of forests, and documents about cardiology lie closer to each other than to documents about tax law. Relative to the uniform background, in which everything appears equally far away, the members of a cluster stand out as outliers with clearly smaller distances. This is why vector search works at all in high dimensions, and why the approximate methods of the next section can exploit the structure of the data.

Within such a cluster, however, the loss of contrast remains. The nearest neighbor and the next few points lie at almost the same distance from the query. The exact nearest neighbor is not necessarily the best match in the sense the user cares about; it is one good match among several that are about equally close. Spending a full scan, or a visit to every leaf, to determine which of them is nearest buys a distinction that carries little meaning.

The dot product: losing direction

Embeddings are usually compared with the dot product or cosine, so we repeat the argument for similarity scores. In 2D, it seems reasonable to assume that a query and its best matches lie in the same quadrant. In Figure 6.19, the query $\boldsymbol{q}$ points to the upper right. Under cosine similarity, $\boldsymbol{d}$ has the highest score because it has the smallest angle to $\boldsymbol{q}$. Under the dot product, $\boldsymbol{e}$ wins, because no other point lies beyond the hyperplane through $\boldsymbol{e}$ orthogonal to $\boldsymbol{q}$. Both best matches lie in the query's quadrant.

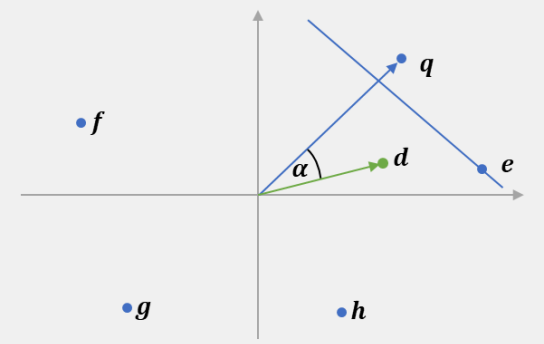


Figure 6.19: A query $\boldsymbol{q}$ and candidate points in 2D: the best matches under cosine ($\boldsymbol{d}$) and dot product ($\boldsymbol{e}$) lie in the query's quadrant.

The high-dimensional counterpart of a quadrant is an **orthant**: the region in which every component has a fixed sign. Two dimensions have 4 quadrants; d dimensions have 2^d orthants. Two vectors share an orthant if their components agree in sign in every dimension. For random vectors, each sign agrees with probability $1/2$, so two vectors share an orthant with probability 2^{-d}.

The number of orthants soon outgrows any collection. With one billion vectors, there are on average about one million vectors per orthant at $d = 10$, but fewer than one at $d = 30$, and about 8×10^{-22} at $d = 100$. Most orthants are empty, and almost no two vectors share one. "The same direction as the query" no longer describes any stored vector.

Conceptually, this is not yet a problem. Each dimension of an embedding can be read as a latent concept, and two vectors that agree on most concepts are similar even if they disagree on a few. Agreement in many dimensions is good enough. The trouble lies in how much "many" still stands out.

Dot products concentrate around zero

Figure 6.20 shows the dot product between pairs of random unit vectors. At $d = 2$, the values spread over the full range from −1 to 1. As the dimension grows, they concentrate around 0, and at $d = 8192$ they form a narrow spike: two random vectors in high dimensions are almost orthogonal.

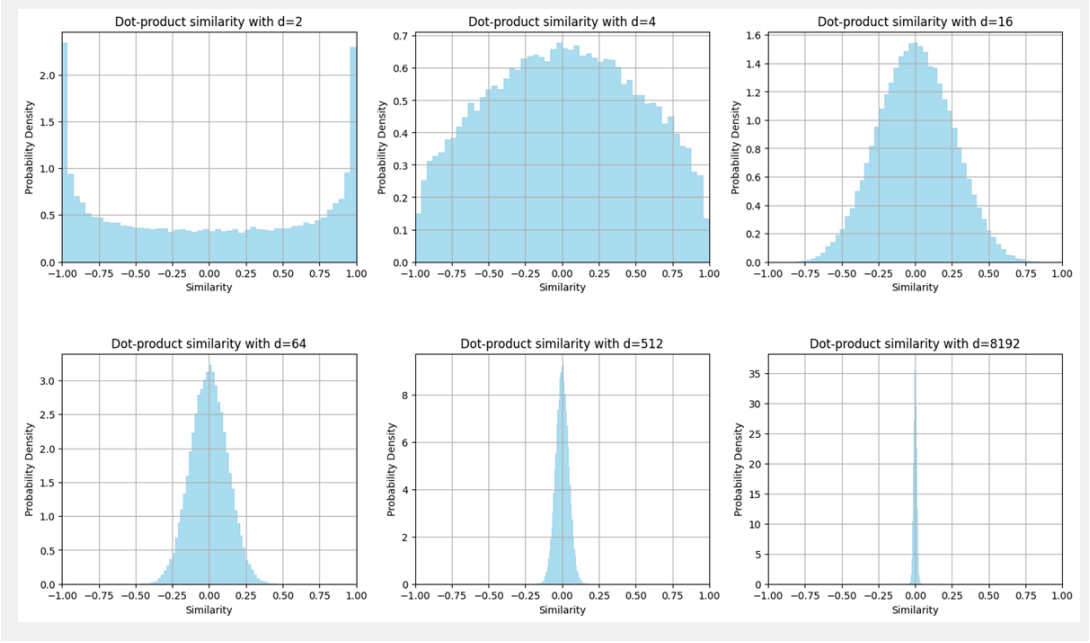


Figure 6.20: Dot products between random unit vectors, from $d = 2$ to $d = 8192$.

A short statistical argument explains the spike. Draw the components x_k and y_k of two vectors independently and uniformly from $[-1, 1]$. Each product $x_k y_k$ has expected value 0 and variance $1/9$. The dot product is the sum of d such products, so it has expected value 0 in any dimension, and its standard deviation grows like $\sqrt{d}/3$. The lengths of the two vectors, however, grow faster: each is about $\sqrt{d/3}$, so their product is about $d/3$. Dividing the two gives the cosine, whose spread shrinks with the dimension.

Key Formula: Concentration of Cosine Similarity

For two independent random vectors in d dimensions, the cosine similarity has expected value 0 and a standard deviation of about

$$\sigma_{\cos} \approx \frac{1}{\sqrt{d}} \tag{6.11}$$

The positive and negative products cancel out more and more completely; random vectors become almost orthogonal.

What does this mean for the dot product? For unit vectors, which most embedding models produce and which Figure 6.20 uses, the dot product is the cosine, and nothing changes. For vectors that are not normalized, the spread of the dot product grows like $\sqrt{d}/3$, so in absolute numbers it does not shrink. But it must be read against the scale of the vectors: the largest possible dot product of two vectors is the product of their lengths, about $d/3$. Relative to this scale, the spread is $\left(\sqrt{d}/3\right)/(d/3) = 1/\sqrt{d}$, exactly as for the cosine.

The lengths themselves concentrate as well. In our example, the length of a random vector is about $\sqrt{d/3}$ with a spread of about 0.26 in every dimension, so its relative variation shrinks like $1/\sqrt{d}$: about 4.5 percent at $d = 100$ and 1.6 percent at $d = 768$. The dot product is the cosine multiplied by two almost equal lengths,

and it ranks random vectors almost exactly like the cosine. In a simulation with 20,000 pairs, the correlation between the two measures is 0.88 at $d = 2$, 0.998 at $d = 100$, and 0.9997 at $d = 768$. The length effect that separates the dot product from the cosine in 2D fades, and the concentration applies to both measures. Learned embeddings may still carry some information in their lengths, but the random background against which a good match must stand out concentrates in the same way.

For the 768-dimensional embeddings of the previous chapter, $\sigma_{\cos} \approx 1/\sqrt{768} = 0.036$; at $d = 8192$, it is 0.011, and the values that separate good from bad matches differ in the third decimal. Even the best of one million random vectors reaches a cosine of only about 0.17 at $d = 768$.

The discussion now runs parallel to the one for distances. Vectors that live alone in their orthants are hard to group, because no two of them agree in all signs, and the loss of contrast again weakens the meaning of similarity. Real data again saves the method. The embeddings of related documents form clusters, and their cosine values stand far above the narrow band around zero that random pairs occupy. A relevant document is an outlier in the distribution of scores. But among these outliers, the top few scores lie close together, and the document with the highest score is not necessarily the best semantic match. Any of the top-k results may be an equally good, or even better, answer.

Good matches instead of the best match

High-dimensional space does not behave like the plane on which our intuition was trained. Distances grow, their contrast fades, and directions become meaningless; index structures that prune well in 2D end up visiting every leaf. At the same time, the loss of contrast questions the goal itself. When the nearest neighbor is only marginally nearer than its competitors, chasing the exact nearest neighbor is wasted effort. What matters is to find good matches: points that clearly stand out from the uniform background, even if they are not provably the nearest.

The retrieval pipelines of Building Semantic Search relax the requirement further. A fast retriever based on embeddings collects candidates, and a reranker reorders them with a more precise but more expensive model. The retriever does not need to deliver the exact top-k in the exact order, as long as the good hits are among its candidates; the reranker restores the precision. Exact nearest-neighbor search is therefore rarely needed. Giving up exactness on purpose, and searching approximately for good matches, buys back the speed that the curse of dimensionality took away. This is the subject of the next section.

Hands-on: The curse of dimensionality

Watch distance distributions collapse in a Monte Carlo experiment, measure how the contrast between the nearest and the farthest point shrinks with the dimension, and explore the escaping-sphere puzzle and the thin outer shell of high-dimensional space. Open notebook ->

Includes pre-run results: you can read through or download and experiment.

Approximate Nearest-Neighbor Search

A retrieval system rarely shows the raw output of a vector search to its users. In Semantic Similarity with Transformers, embeddings served as a first stage: a bi-encoder retrieves a few hundred candidates, and a cross-encoder reorders them with a stronger and more expensive model. Media retrieval follows the same pattern. A compact color feature finds candidate images, and a more detailed comparison puts them in their final order. For such a first stage, the exact nearest neighbor has no special value. What matters is that the good matches are among the candidates, because the second stage decides their order.

Even without a second stage, users want to see good matches, not necessarily the single best one. The previous section showed that high dimensions reduce contrast: the nearest neighbor is only marginally closer than the next few points, and all of them stand out clearly from the background. Returning the third-best match instead of the best one hardly changes what the user sees. This opens the door to a trade: we give up exactness and gain speed.

What carries over from the previous section

- **High dimensions defy 2D intuition**: distances grow like $\sqrt{d}$, while their spread stays the same. The contrast between the nearest and the farthest point vanishes, $(\delta_{\max} - \delta_{\min})/\delta_{\min} \to 0$, and dot products of random vectors concentrate around zero. This holds for Euclidean distance, dot product, and cosine alike.
- **Trees can no longer prune**: beyond a few dozen dimensions, the sphere around the query that passes through its nearest neighbor intersects every leaf, so even the optimal search algorithm reads all the data. In practice, a full scan is faster than a tree index beyond roughly ten dimensions.
- **Real data still works**: related items form clusters, and good matches stand out clearly from a background in which everything else appears equally far away.

For the full background, see The Curse of Dimensionality.

How can we accelerate the search? A full scan reads every vector, so its cost is the number of vectors times the size of each vector. A concrete collection makes this tangible. One billion embeddings with 768 dimensions, stored as 4-byte floats, occupy $10^9 \times 768 \times 4$ bytes, about 3 TB. A full scan reads all 3 TB for every query. There are three levers to reduce this volume, and this section treats them in turn:

1. **Fewer dimensions**: map the vectors into a lower-dimensional space.
2. **Fewer bits per vector**: store each vector as a compact code.
3. **Fewer vectors read**: visit only the part of the collection near the query.

Each lever trades a little accuracy for speed. Combined, they reduce the work per query by two to three orders of magnitude.

Fewer dimensions

The first lever shrinks every vector before the search starts. **Principal component analysis** (PCA) finds the directions along which the data varies most and projects every vector onto the leading ones. It works well when the original dimensions are not independent. The color histogram of Distance and Similarity Measures is a good case. Neighboring bins cover similar intensities, so a slightly brighter photograph moves pixels from one bin into the next. Correlated dimensions carry overlapping information, and a few principal components capture most of it. Distances in the reduced space then approximate the distances in the original space well.

Embeddings can be trained for this purpose. The Matryoshka embeddings of Building Semantic Search arrange their information so that the leading dimensions carry the most significant part, and a prefix of the vector is itself a usable embedding. Truncation needs no projection at all, only a shorter read.

Dimensionality reduction is a modest lever. Depending on the features, it removes roughly 20 to 60 percent of the data without changing the results much. Reducing our example collection from 768 to 384 dimensions halves 3 TB to 1.5 TB. Every vector must still be read, so the search remains a full scan, only a cheaper one.

Fewer bits per vector

The second lever keeps the dimensions but stores each vector with fewer bits. A float uses 32 bits per component, far more precision than a similarity comparison needs. The errors that compression introduces are small compared to the gap between good matches and the background, so they rarely change which matches stand out.

Scalar quantization

The simplest form replaces each float with a small integer. We divide the value range of each dimension into 2^b intervals and store only the number of the interval that contains the value. With $b = 8$, every component becomes one byte, and the vector shrinks by a factor of 4. With $b = 4$, as in the FAISS index type `SQ4`, it shrinks by a factor of 8. Distances computed on the interval numbers are approximations of the true distances.

Quantization does not have to give up exactness. The **vector approximation file** (VA-file) of Weber, Schek, and Blott (1998) accepts the conclusion of the previous section: if every leaf must be read anyway, make reading cheaper. The interval numbers of a vector describe a cell of a grid, and from the cell we can compute a lower and an upper bound on the distance between the query and any vector inside it. The search first scans the compact approximations and discards every vector whose lower bound exceeds the k-th smallest upper bound. Only the few remaining candidates are read in full and compared exactly. The result is exact, while the scan reads only a fraction of the data. Skipping the exact comparisons turns the same codes into an approximate method. Scalar quantization is therefore the bridge between exact and approximate search.

How the VA-file stays exact (optional reading)

Consider two dimensions with values in $[0, 1]$ and $b = 2$ bits per dimension. Each dimension splits into four intervals of width 0.25, and every vector is stored as two 2-bit numbers. The vector $\boldsymbol{p} = (0.62, 0.10)$ lies in interval 2 of the first dimension, $[0.50, 0.75]$, and in interval 0 of the second, $[0, 0.25]$, so its approximation is the code $(10, 00)$.

For the query $\boldsymbol{q} = (0.20, 0.55)$, the code alone bounds the distance. In the first dimension, any value in $[0.50, 0.75]$ lies at least $0.50 - 0.20 = 0.30$ and at most $0.75 - 0.20 = 0.55$ from q_1. In the second dimension, any value in $[0, 0.25]$ lies at least $0.55 - 0.25 = 0.30$ and at most $0.55 - 0 = 0.55$ from q_2. Combining the two dimensions gives $\sqrt{0.30^2 + 0.30^2} \approx 0.42$ as the lower bound and $\sqrt{0.55^2 + 0.55^2} \approx 0.78$ as the upper bound. The exact distance, $\sqrt{0.42^2 + 0.45^2} \approx 0.62$, lies between them.

The search runs in two phases. The filtering phase scans all approximations, computes both bounds for each, and keeps track of the k smallest upper bounds. A vector whose lower bound exceeds the k -th smallest upper bound cannot be among the k nearest neighbors and is discarded. The refinement phase visits the remaining candidates in order of increasing lower bound, reads their exact vectors, and computes their true distances. It stops as soon as the next lower bound exceeds the k-th smallest true distance found so far, because no later candidate can be closer.

Both phases are exact, so the result is exact. The gain comes from the size of the approximations: with 4 to 8 bits per dimension, the filtering phase reads 4 to 8 times less data than a full scan, and only a small number of exact vectors is read in the refinement phase. Because the scan is sequential, it also avoids the random disk accesses of a tree index.

Product quantization

Scalar quantization treats every dimension separately and needs at least one bit per dimension. **Product quantization** (PQ), introduced by Jégou, Douze, and Schmid (2011), quantizes groups of dimensions together. Figure 6.21 shows the idea for the eight-dimensional vector $(0.1, 0.3, 0.5, 0.2, 0.6, 0.4, 0.9, 0.8)$. We split it into $m = 4$ sub-vectors of two components each: $(0.1, 0.3)$, $(0.5, 0.2)$, $(0.6, 0.4)$, and $(0.9, 0.8)$. Each sub-space has its own **codebook** of 2^b representative sub-vectors, found by running k-means on the sub-vectors of the collection. With $b = 2$, each codebook holds four centers, and each sub-vector is

replaced by the 2-bit number of its nearest center: 00, 10, 01, and 10. The code 10 appears twice but refers to different centers, because sub-spaces 2 and 4 have different codebooks. Instead of eight floats with 256 bits, the vector is stored in 8 bits, a compression of 1:32.

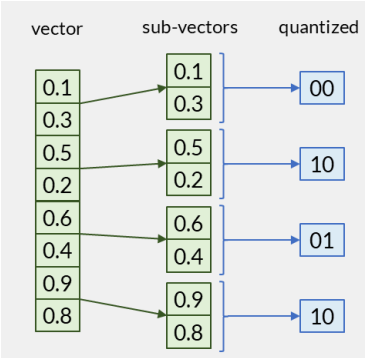


Figure 6.21: Product quantization of an eight-dimensional vector. The vector is split into four sub-vectors, and each sub-vector is replaced by the 2-bit number of its nearest center in its own codebook. Eight floats become 8 bits.

The search uses lookup tables. We split the query $\boldsymbol{q}$ in the same way into $\boldsymbol{q}_1, \ldots, \boldsymbol{q}_m$ and compute, for every sub-space j and every center of its codebook, the squared distance between $\boldsymbol{q}_j$ and that center. This fills m tables with 2^b entries each, once per query. The approximate distance to a stored vector is then a sum of m table look-ups, one per code.

Key Formula: Product Quantization Distance

$$\tilde{\delta}(\boldsymbol{q}, \boldsymbol{p})^2 = \sum_{j=1}^{m} \| \boldsymbol{q}_j - \boldsymbol{c}_j(\boldsymbol{p}) \|^2 \tag{6.12}$$

Here $\boldsymbol{c}_j(\boldsymbol{p})$ is the center of codebook j that represents the j-th sub-vector of $\boldsymbol{p}$. The query stays exact; only the stored vector is replaced by its centers, so each term is read from a table computed once per query.

Example

For our collection of one billion 768-dimensional vectors, we choose $m = 96$ sub-vectors of 8 dimensions each and $b = 8$ bits per code. Every vector becomes 96 bytes instead of 3,072, a compression of 1:32, and the whole collection needs 96 GB instead of 3 TB. Per query, the lookup tables hold 96×256 entries, each a distance over 8 dimensions: $96 \times 256 \times 8 \approx 197,000$ multiply-adds, as much as comparing the query with 256 full vectors. Against a billion stored vectors, this cost is negligible. Each stored vector then costs 96 look-ups and additions instead of 768 multiply-adds, and 96 bytes read instead of 3,072.

The standard choice is $b = 8$: each code is one byte, and each table of 256 entries fits into the processor cache. The sub-vector length then sets the compression. Sub-vectors of 4 to 16 dimensions give ratios between 16 and 64 times over 4-byte floats. Jégou and colleagues stored 128-dimensional SIFT image descriptors in 8 bytes, a ratio of 64. Longer sub-vectors compress more but approximate the distance more coarsely, which a final reranking step on the exact vectors compensates. At 96 bytes for 768 dimensions, PQ uses exactly one bit per dimension, and at 48 bytes only half a bit, a rate that scalar quantization cannot reach.

Locality-sensitive hashing

Locality-sensitive hashing (LSH), one of the first approximate methods from the late 1990s, maps vectors to bit strings. It chooses h hyperplanes and records for each one on which side a vector lies: bit j is 1 if the vector lies on the positive side of hyperplane j and 0 otherwise. In Figure 6.22, three hyperplanes divide the plane into seven regions. One of the eight possible 3-bit codes does not occur, because three lines cannot create more than seven regions in the plane. The query $\boldsymbol{q}$ lies on the positive side of hyperplane 1 only, so

its code is 100. Points in the same region share the query's code, and points in a neighboring region differ from it in one bit.

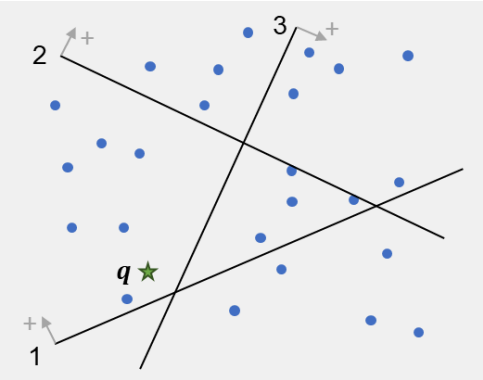


Figure 6.22: Locality-sensitive hashing with three hyperplanes. Each point receives one bit per hyperplane, set to 1 on the positive side. The query $\boldsymbol{q}$ lies on the positive side of hyperplane 1 only and receives the code 100.

The obvious way to use the codes is as hash buckets: return the points that share the query's code. This fails as soon as h grows. With 64 hyperplanes there are 2^{64} possible codes, far more than stored vectors, so most buckets are empty, and the query's bucket is likely empty as well. LSH therefore ranks all stored codes by their **Hamming distance** to the query's code, the number of bits in which they differ, and returns the vectors with the smallest Hamming distance. Comparing bit strings is extremely cheap: an exclusive-or followed by a count of the set bits. With $h = 256$ hyperplanes, a 768-dimensional vector shrinks from 3,072 bytes to 32 bytes, a factor of 96. Since the Hamming distance takes only $h + 1$ different values, many vectors tie, and an exact reranking of the best candidates resolves the ties.

The Hamming distance is a good proxy because it tracks the angle between vectors. For random hyperplanes through the origin, the probability that a hyperplane separates two vectors equals θ/π, where θ is the angle between them, so the expected Hamming distance is $h \cdot \theta/\pi$. The hyperplanes in the figure are drawn freely to make the regions visible; the analysis assumes hyperplanes through the origin, the common choice in practice. Because LSH approximates the angle, it serves cosine similarity directly. For the dot product, the extension trick of Low-Dimensional Search Structures first gives all stored vectors the same length, after which ranking by angle equals ranking by dot product.

Why the Hamming distance tracks the angle (optional reading)

Consider two vectors $\boldsymbol{x}$ and $\boldsymbol{y}$ with angle θ between them, and a random hyperplane through the origin. The hyperplane separates the two vectors if they fall on different sides, which gives them different bits. Only the plane spanned by $\boldsymbol{x}$ and $\boldsymbol{y}$ matters for this question. The hyperplane cuts this plane along a line through the origin, and for a random hyperplane the direction of this line is uniformly distributed.

The line separates $\boldsymbol{x}$ from $\boldsymbol{y}$ exactly if it passes through the angle between them. A line through the origin covers two opposite directions, so it falls into the angle θ or into its mirror image on the other side of the origin. Both together cover 2θ of the full circle of 2π, and the probability of a separating hyperplane is $2\theta/2\pi = \theta/\pi$. This argument is due to Charikar (2002), who introduced hyperplane hashing for the cosine.

With h independent hyperplanes, the Hamming distance is a count of h independent events with probability θ/π each. Its expected value is $h \cdot \theta/\pi$, and its standard deviation is $\sqrt{h \cdot (\theta/\pi)(1 - \theta/\pi)}$. For $h = 256$ and $\theta = \pi/4$, the expected Hamming distance is 64 with a standard deviation of about 6.9. The estimate of θ/π is therefore accurate to about ± 0.03, and more bits make it more accurate.

The same reasoning explains the limits of LSH. Two vectors at angles of 0.24π and 0.26π to the query are hard to tell apart with 256 bits, which is exactly the situation inside a cluster, where the nearest neighbors are almost equally close. LSH finds the cluster reliably but not the order within it. This is why LSH results are usually reranked with exact vectors.

The second lever gains between a factor of 4 for 8-bit scalar quantization and roughly a factor of 100 for short LSH codes, with product quantization typically in between at 16 to 64. Every vector is still read, but each one costs only a fraction of a full comparison.

Fewer vectors read

The third lever reduces the number of vectors read. This was the idea behind the tree structures in Low-Dimensional Search Structures: skip regions that cannot contain the answer. The previous section showed that exact pruning fails in high dimensions, because the sphere around the query intersects every region. Approximate methods relax the requirement. They visit only the regions most likely to contain good matches and accept that a neighbor just across a boundary may be missed. Figure 6.23 contrasts the two approaches.



Figure 6.23: Exact search compares the query to every vector (left). An approximate method partitions the space into clusters and searches only those near the query (right), trading a little recall for a large speed-up.

Inverted file

The **inverted file** (IVF) borrows the inverted index from text retrieval. A clustering algorithm, usually k-means, chooses n_{list} centers and assigns every vector to its nearest center, as in Figure 6.24. The index is a dictionary that maps each center to the list of its vectors, just as an inverted index maps each term to its posting list. A query is first compared with all centers, and the search then scans only the lists of the n_{probe} nearest centers. The share of the collection read is $n_{\text{probe}}/n_{\text{list}}$. For our billion vectors with $n_{\text{list}} = 4,096$, each list holds about 244,000 vectors, and probing 16 lists reads $1/256$ of the collection.

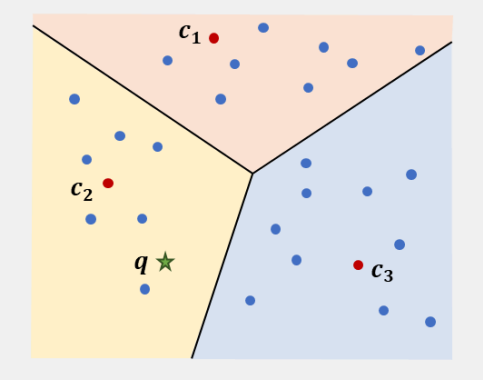


Figure 6.24: An inverted file clusters the data around centers. The query is routed to its nearest center c_2, and only that cluster's list is scanned; probing more clusters trades speed for recall.

The parameter n_{probe} is the tuning knob. A neighbor that sits just across a cluster boundary is missed if its list is not probed, and each additional list recovers some of these misses. The cost grows linearly with the number of probed lists, while the gain in recall shrinks. The following measurements from the ann-benchmarks project, discussed below, use FAISS with 4,096 lists on 1.2 million GloVe word vectors with 100 dimensions:

Probed lists	Share read	Recall@10	Queries per second
5	0.1%	0.59	4,929
10	0.2%	0.68	3,443

50	1.2%	0.84	1,003
100	2.4%	0.89	548
200	4.9%	0.93	298

Doubling the probes from 100 to 200 almost halves the throughput but adds only 4 points of recall. Even at 200 probes, the search is 18 times faster than the full scan, which manages 16.5 queries per second on the same hardware.

Navigable small-world graphs

A different approach navigates a graph. A **navigable small-world** (NSW) graph turns every vector into a node and links it to a fixed number of its nearest neighbors, its "friends", plus some long-range links that act as shortcuts across the space. The graph has two key properties. It is **navigable**: from any node, a step to the neighbor closest to the query makes progress toward the query. And it is a **small world**: any two nodes are connected by a short path, whose length grows only poly-logarithmically with the number of nodes.

Search starts at an entry point and greedily moves to whichever neighbor is closest to the query, as traced in Figure 6.26. It stops when no neighbor is closer than the current node. This node is a local minimum of the distance on the graph, not necessarily the global one, which is why the result is approximate. To reduce the risk of stopping early, the search keeps a list of the best ef_{search} candidates instead of a single node and expands all of them. A longer list costs more steps and finds better neighbors.

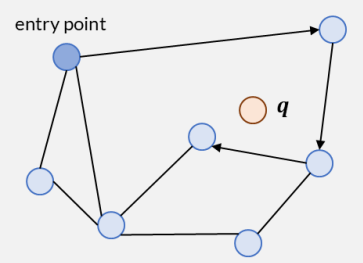


Figure 6.26: Greedy search on a navigable small-world graph. From the entry point, each step moves to the neighbor closest to the query q until no neighbor is closer.

The **hierarchical** version, HNSW (Malkov and Yashunin, 2016), stacks several such graphs. The base layer contains all vectors. Each higher layer contains a random subset of the layer below, with fewer and fewer nodes toward the top. The search starts at the top, where few nodes and long links cover large distances in few steps. It then descends layer by layer, using the best node of each layer as the entry point for the next one, and refines the result on the base layer. Two parameters control the index: M, the number of links per node, and ef_{search}, the length of the candidate list. HNSW is among the fastest ANN methods and is included in most vector databases.

The speed has a price. The links need memory: with $M = 32$, the base layer stores about 64 links of 4 bytes per node, roughly 256 bytes per vector. Greedy steps jump to arbitrary nodes, so the vectors must reside in main memory. For our billion vectors, this means 3 TB of vectors plus 256 GB of links in memory. Building the graph is also slow, because every insertion runs a search to find the new node's friends.

The graph and the inverted file combine well. Instead of single vectors, the base layer can hold clusters: the graph is built over the IVF centers, and each center points to its list. This matters when the number of lists is large. With $n_{\text{list}} = 65,536$ lists for a billion vectors, comparing the query with every center already becomes noticeable, while a graph over the centers finds the nearest ones in a few steps. The search then scans the lists of the n_{probe} nearest centers, and its cost again grows linearly with the number of lists visited. In FAISS, the coarse quantizer `IVF65536_HNSW32` implements this design. It also suits collections larger than main memory: the graph over the centers stays in memory, while the lists reside on disk.

Combining the levers

No single lever wins on its own, so production libraries combine all three. Facebook AI Similarity Search (FAISS) is the standard example. It organizes an index into four stages, which map directly onto the levers:

- **Vector transformers** prepare vectors: PCA to reduce dimensions (lever 1), or L2-normalization for cosine search.
- **Coarse quantizers** select the vectors to read, for example with IVF or HNSW (lever 3).
- **Fine quantizers** encode the selected vectors compactly, for example with scalar quantization, PQ, or LSH (lever 2).
- **Refiners** rerank the best k' candidates with exact distances on the original vectors, or apply metadata filters.

A FAISS index type names its stages in this order. For our billion vectors, `PCA384,IVF4096,PQ48,RFlat` reduces the vectors to 384 dimensions, clusters them into 4,096 lists, encodes each vector into 48 bytes, and reranks the best candidates exactly. The codes of the whole collection take 48 GB and fit into main memory. Probing 16 lists reads $10^9/256 \approx 3.9$ million codes of 48 bytes each, about 190 MB per query. Reranking $k' = 100$ candidates reads another 300 KB of exact vectors from disk. Compared with the 3 TB of a full scan, the data read per query shrinks by a factor of about 16,000. The actual speed-up is smaller, because table look-ups, center comparisons, and random reads also cost time.

The levers compound their errors as well as their gains. PCA loses some information, PQ distorts the distances, and IVF misses neighbors in unprobed lists. Each error alone is harmless, but stacking several approximations can reduce result quality quickly. The parameters, here the target dimension, the code length, n_{list}, n_{probe}, and k', must therefore be tuned together, by measuring the result quality on a sample of queries, ideally against relevance judgments rather than only against exact search.

Measuring the trade-off

Every ANN method exposes parameters that trade recall for speed: the number of probed lists in IVF, the candidate list length in HNSW, the code length in PQ and LSH. To compare methods fairly, we measure both axes at once. The quality axis is recall against the exact result.

Key Formula: Recall@k for Approximate Search

$$\text{recall@}k = \frac{|\ \text{approximate top-}k \cap \text{exact top-}k\ |}{k} \tag{6.13}$$

The fraction of the true k nearest neighbors that the approximate search also returned; 1.0 means the approximation matched exact search on this query.

The speed axis is throughput, in queries per second. Plotting throughput against recall traces a curve for each method, and the public benchmark at ann-benchmarks.com collects these curves for many libraries on standard datasets. Figure 6.27 shows the comparison on GloVe-100: 1.2 million word vectors with 100 dimensions, compared by cosine, with $k = 10$. Reading it is straightforward: fix the recall you need, then pick the method with the highest throughput at that recall. A method whose curve lies above another one dominates it.

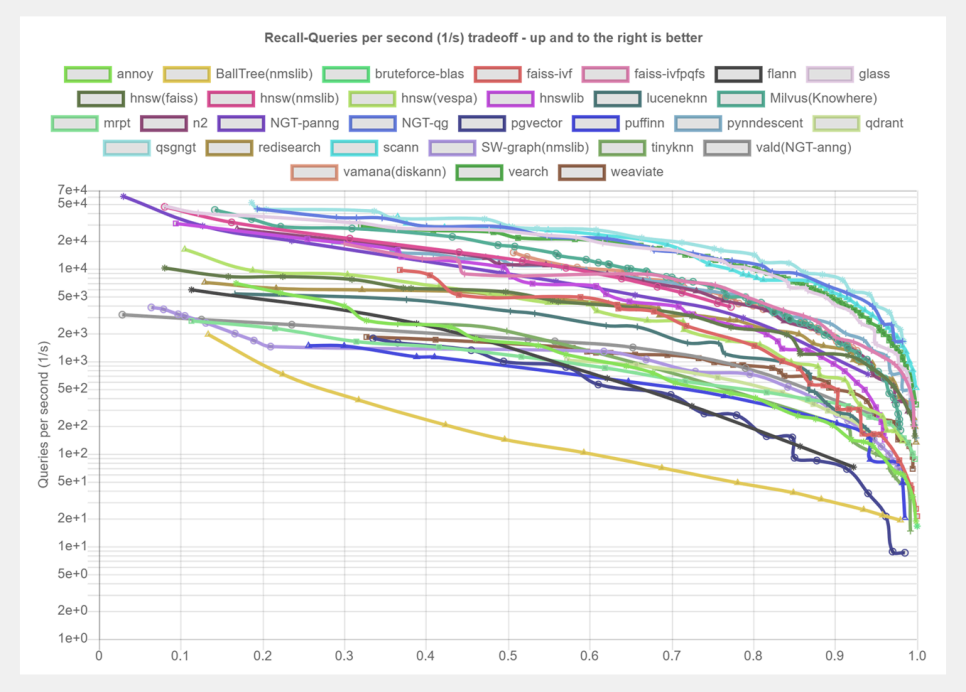


Figure 6.27: Recall versus queries per second for about 30 ANN methods on GloVe-100 ($k = 10$, cosine), from ann-benchmarks. Up and to the right is better; at a chosen recall, the highest curve is the fastest method.

The plot quantifies the trade. The full scan reaches 16.5 queries per second at a recall of 1.0. The fastest method at each recall level does far better:

Recall@10 at least	Best queries per second	Speed-up over full scan
0.80	14,100	850
0.90	7,560	460
0.95	3,980	240
0.99	1,000 to 1,700	60 to 100

Approximation buys two to three orders of magnitude, and the last few points of recall are the most expensive. The plot does not show everything, though. Memory use and build time appear in separate plots on the benchmark site, and they are where methods differ most: HNSW needs its graph and the full vectors in memory, while PQ needs a fraction of the memory and builds quickly.

Why does approximate search work?

The ann-benchmarks view makes approximation look costly. The last points of recall are expensive, and many cheap configurations return only a third of the exact top 10. But recall against exact search compares one retriever with another. A user never sees the exact top 10. The user sees the final result of a pipeline, and its quality is measured against relevance judgments with nDCG, as in Evaluating Graded Relevance.

We test this on TREC-COVID: 171,332 abstracts of COVID-19 research papers, 50 topics, and graded relevance judgments. The retrievers combine the three levers. Matryoshka embeddings from Qwen3-Embedding-0.6B are truncated to between 1024 and 64 dimensions. A full scan, IVF, or HNSW selects the vectors, and exact floats, PQ codes, or LSH bits compare them. This gives 140 retrievers. Each returns its top 50, and the cross-encoder Qwen3-Reranker-0.6B reorders them into the final top 10. TREC judged only the documents that the systems of 2020 retrieved, so the pool from Designing a Retrieval Benchmark misses documents that only an approximate retriever surfaces. An LLM judge graded these missing pairs. Otherwise they would count as not relevant and penalize exactly the retrievers we want to evaluate.

Figure 6.29 measures every retriever in three ways. We focus on the 126 retrievers that read at least 100 times less data than a full scan. Against exact search (left), their median recall@10 is 0.24, so the typical cheap retriever returns only a quarter of the exact top 10. By this measure, approximate search fails. Judged

for relevance (middle), the same top 10 reach a median nDCG@10 of 0.80, against 0.91 for exact search. The documents they return instead of the exact neighbors are often just as relevant. This is the loss of contrast from The Curse of Dimensionality at work: the exact neighbors are only marginally nearer than the next candidates, and many of those candidates are good matches too. After reranking (right), the median rises to 0.88, against 0.92 for exact search, and 44 percent of the cheap retrievers end within 0.03 of exact search.

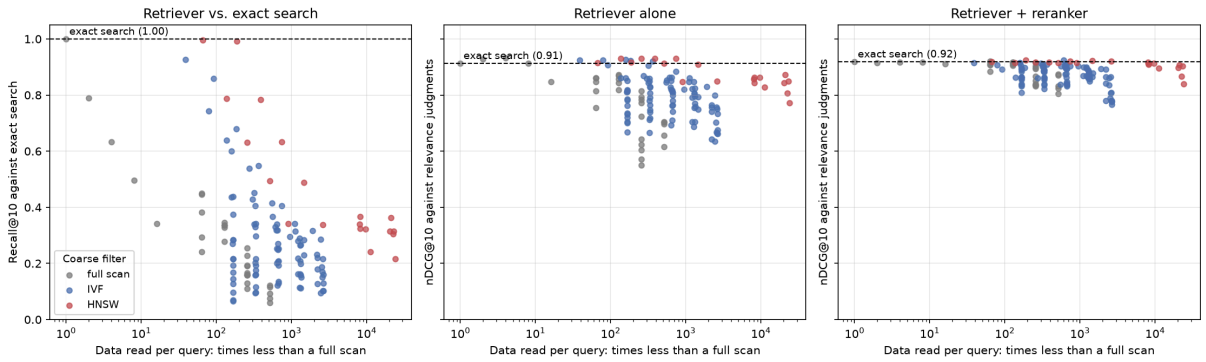


Figure 6.29: Recall against exact search (left), nDCG@10 of the retriever alone (middle), and nDCG@10 after reranking its top 50 (right) for 140 retrievers on TREC-COVID. Each point is one retriever; the dashed line marks exact search.

Figure 6.30 follows five retrievers through the pipeline. One retriever uses 256 dimensions, 16 of 1,024 IVF lists, and 32-byte PQ codes. It reads 608 times less data than a full scan and returns only a third of the exact top 10, yet after reranking it reaches an nDCG@10 of 0.92, the same as exact search. HNSW on 64-dimensional embeddings reads 2,604 times less data with the same result. Once a reranker follows, even aggressive Matryoshka truncation costs almost nothing.

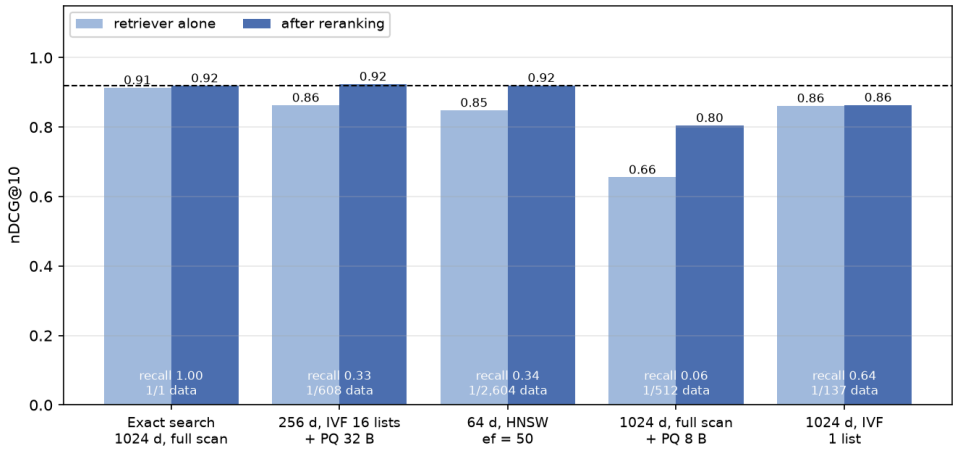


Figure 6.30: nDCG@10 of five retrievers alone and after reranking, annotated with recall@10 against exact search and the share of data read.

The two remaining retrievers show where this stops. PQ with 8 bytes per vector distorts the distances so strongly that only 6 percent of the exact top 10 survive, and on its own it reaches an nDCG@10 of 0.66. Many relevant documents are still among its 50 candidates, so the reranker lifts the result to 0.80 and repairs more than half of the gap. IVF with a single probed list fails differently. It never reads the clusters next to the query's own cluster, so the reranker never sees their documents, and the result stays at 0.86 before and after reranking. **Fine-filter errors are repairable; coarse-filter errors are not.** Production systems therefore keep the coarse filter generous, with enough probed lists or a long candidate list, and let the fine filter be aggressive.

There is no single best method, only the best operating point for a given collection, memory budget, and quality target. Recall against exact search is a useful diagnostic for one index, but the operating point should be chosen on end-to-end quality: the cheapest retriever whose reranked result stays close to that of exact search. The benchmark above covers one collection and one embedding model, so the operating point must be measured again for each application. HNSW is a strong default as long as the vectors fit into main memory. Beyond that, IVF with product quantization and a final reranking step is the common choice.

Hands-on: Approximate nearest-neighbor search

Build simplified versions of IVF, LSH, and product quantization, watch greedy search navigate an HNSW graph, and compare FAISS index configurations on the recall-throughput frontier. Open notebook ->

Includes pre-run results: you can read through or download and experiment.

Hands-on: Benchmarking approximate search with a reranker

Combine Matryoshka truncation, a coarse filter (IVF or HNSW), and a fine filter (PQ or LSH) into retrievers, rerank their candidates with a cross-encoder, and see why recall against exact search understates the quality of the final ranking. Open notebook ->

Includes pre-run results: you can read through or download and experiment.

Vector Search in Practice

The methods of the previous section rarely reach an application in raw form. They arrive packaged inside libraries, search engines, and databases that also handle storage, updates, and scale. This section follows one realistic query through four widely used systems: the FAISS library, the Lucene library, the OpenSearch engine, and the pgvector extension for PostgreSQL. We first look at how each system answers the vector part of the query, and then at how each one adds metadata filters and keyword search.

A running example: movie search

The movie table of Practical Frameworks stored a title, an overview, a rating, and a year for every movie. We extend it with two text fields, the main actors and the genres, and with an embedding of the overview, computed by an embedding model as in Building Semantic Search. With a model such as Qwen3-Embedding-0.6B, every movie receives a vector with 1,024 dimensions. A user now types:

> a mind-bending heist thriller with Leonardo DiCaprio, released after 2005, rated 7.5 or better

No single retrieval method covers this request. Each part of it asks for a different kind of match:

Part of the query	Kind of match	Handled by
“mind-bending heist”	meaning of the plot	vector search on the overview embedding
“with Leonardo DiCaprio”	exact name	BM25 on the actors field
“thriller”	category term	BM25 on the genres field
“released after 2005”, “rated 7.5 or better”	hard constraints	filters on year and rating

The overview of *Inception* describes a thief who steals secrets by entering the dreams of his targets. It contains neither “mind-bending” nor “heist”, so BM25 on the overview would miss it. An embedding does better, but not well enough on its own: among 10,000 movies embedded with Qwen3-Embedding-0.6B, the vector search alone ranks Inception only 138th, behind movies whose overviews match “heist thriller” more literally. Names behave the other way round. An embedding blurs “Leonardo DiCaprio” into a region of similar actors, while BM25 matches the name exactly. The year and the rating are not matters of degree at all: a movie from 2001 must not appear, however similar it is. Each ingredient matters. The filters on year and rating lift Inception to second place among the 261 admitted movies, and fusing the vector ranking with BM25 on the actors puts it first. The rest of this section builds this query step by step: first the vector search, then the filters and the keyword search.

Vector search in four systems

FAISS: a library of vector indexes

FAISS (Facebook AI Similarity Search), developed at Meta, is a C++ library with Python bindings that runs on CPUs and GPUs. It implements all methods of the previous section and lets us combine them freely through index type strings. FAISS stores only vectors and their integer identifiers. The titles, the actors, and the years stay in the application’s own data store, and the application maps the returned identifiers back to movies.

```
import faiss
import numpy as np

# 1. Movie embeddings (one row per movie) and their movie ids
X = np.load("overview_embeddings.npy").astype("float32")   # shape (n, 1024)
ids = np.load("movie_ids.npy").astype("int64")
faiss.normalize_L2(X)               # unit length: inner product = cosine
```

```
# 2. Build an IVF index with PQ codes (lever 3 + lever 2)
index = faiss.index_factory(1024, "IVF256,PQ64", faiss.METRIC_INNER_PRODUCT)
index.train(X)                       # k-means for the centers and the codebooks
index.add_with_ids(X, ids)

# 3. Search: embed the query and probe 16 of the 256 lists
q = embed("a mind-bending heist thriller").reshape(1, -1)
faiss.normalize_L2(q)
index.nprobe = 16
scores, movie_ids = index.search(q, 10)
```

Step 1 normalizes the vectors, so that the inner product equals the cosine similarity. Step 2 builds the composite index of the previous section: `train` runs k-means to find the 256 centers of the inverted file and the codebooks of the product quantizer, and `add_with_ids` encodes every vector into 64 one-byte codes. Step 3 sets the tuning knob n_{probe} and returns the ten best movie ids with their approximate scores. Changing the index string, for example to `HNSW32` or to `IVF256,PQ64,RFlat` with an exact refinement stage, changes the method without changing the rest of the code.

Lucene: vectors next to the inverted index

Lucene stores vectors as one more field type of a document. The document of Practical Frameworks receives a `KnnFloatVectorField` next to its text and numeric fields:

```
// 1. A movie document with text, numeric, and vector fields
Document doc = new Document();
doc.add(new TextField("title", "Inception", Field.Store.YES));
doc.add(new TextField("actors", "Leonardo DiCaprio, Joseph Gordon-Levitt",
Field.Store.NO));
doc.add(new TextField("genres", "Thriller, Science Fiction", Field.Store.NO));
doc.add(new IntField("year", 2010, Field.Store.YES));
doc.add(new FloatField("rating", 8.8f, Field.Store.YES));
doc.add(new KnnFloatVectorField("embedding", embedding,     // float[1024]
    VectorSimilarityFunction.COSINE));
writer.addDocument(doc);

// 2. Approximate k-NN query on the vector field
float[] query = embed("a mind-bending heist thriller");
Query knn = new KnnFloatVectorQuery("embedding", query, 10);
TopDocs top = searcher.search(knn, 10);
```

Lucene builds an HNSW graph for the vector field of every segment. A query searches the graph of each segment and merges the per-segment results, the same scatter-gather pattern as for text queries. Because segments are merged in the background, their graphs must be merged as well, which makes indexing vectors considerably more expensive than indexing text. Lucene reports cosine similarity as a score in $[0, 1]$, computed as $(1 + \cos)/2$, so that higher always means better. For lever 2, Lucene offers vector formats that quantize the stored vectors to 7 or 4 bits per component, or to a single bit. The full vectors stay in the index, so the best candidates can be rescored with them: since version 10.3, the `RescoreTopNQuery` does this inside Lucene; in earlier versions, the application rescores.

OpenSearch: distributed vector search

OpenSearch wraps Lucene in a distributed engine, as described in Scaling Out. Vector search is a field type of the index mapping. The `knn_vector` field declares the dimensions, the similarity, and the ANN method:

```
# 1. Index mapping: text fields, numeric fields, and a vector field
client.indices.create(index="movies", body={
    "settings": {"index": {"knn": True}},
    "mappings": {"properties": {
        "title":   {"type": "text"},
```

```
        "actors":  {"type": "text"},
        "genres":  {"type": "text"},
        "year":    {"type": "integer"},
        "rating":  {"type": "float"},
        "embedding": {
            "type": "knn_vector",
            "dimension": 1024,
            "space_type": "cosinesimil",
            "method": {"name": "hnsw", "engine": "faiss",
                       "parameters": {"m": 16, "ef_construction": 128}},
        },
    }},
})

# 2. Approximate k-NN query
query = embed("a mind-bending heist thriller")
response = client.search(index="movies", body={
    "size": 10,
    "query": {"knn": {"embedding": {"vector": query, "k": 10}}},
})
```

The `engine` parameter selects the implementation behind the field. The Lucene engine uses Lucene's own HNSW graphs; the FAISS engine runs FAISS inside every shard and also offers IVF and product quantization. The parameters `m` and `ef_construction` are the HNSW parameters of the previous section. A query is sent to every shard, each shard searches its own index and returns its own top k, and a coordinating node merges the shard results. The global top k is always among the per-shard lists, so merging adds no error of its own; the approximation comes only from the ANN search inside each shard. The price is that every query must visit every shard.

pgvector: vectors in SQL

The **pgvector** extension adds a `vector` column type to PostgreSQL. We extend the movie table with a vector column and an HNSW index:

```
-- 1. Enable the extension and add a vector column
CREATE EXTENSION IF NOT EXISTS vector;
ALTER TABLE movies ADD COLUMN embedding vector(1024);

-- 2. Approximate index: HNSW with cosine distance
CREATE INDEX ON movies USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);

-- 3. k-NN query: order by cosine distance to the query embedding
SET hnsw.ef_search = 100;
SELECT title, year, embedding <=> '[0.021, -0.013, ...]' AS distance
FROM movies
ORDER BY embedding <=> '[0.021, -0.013, ...]'   -- 1,024 values
LIMIT 10;
```

pgvector works with distances: smaller is better. The operator `<=>` computes the cosine distance $1 - \cos$, `<->` the Euclidean distance, `<+>` the Manhattan distance, and `<#>` the negative dot product, negated to fit the same convention. Without an index, the query scans all rows and returns exact results. With the HNSW index of step 2, PostgreSQL's planner uses the index for the `ORDER BY ... LIMIT` pattern. The index parameters are `m` (default 16) and `ef_construction` (default 64); the query parameter `hnsw.ef_search` (default 40) sets the candidate list length. The second index type, IVFFlat, is an inverted file with exact vectors in its lists, with `lists` centers and `ivfflat.probes` probed lists (default 1).

pgvector also exposes the other two levers. A `halfvec` column stores 16-bit floats. The function `binary_quantize` reduces every component to its sign bit, compared with the Hamming distance operator

`<~>`. An index on `subvector(embedding, 1, 256)` searches only the leading 256 components of a Matryoshka embedding. The refine step is plain SQL: an inner query retrieves candidates through a compact index, and an outer query reorders them by the full vectors.

```
-- Binary codes for the candidate search, full vectors for the final order
CREATE INDEX ON movies
  USING hnsw ((binary_quantize(embedding)::bit(1024)) bit_hamming_ops);

SELECT title FROM (
  SELECT title, embedding FROM movies
  ORDER BY binary_quantize(embedding)::bit(1024)
           <~> binary_quantize('[0.021, ...]'::vector)
  LIMIT 100                                    -- k' = 100 candidates
) AS candidates
ORDER BY embedding <=> '[0.021, ...]'
LIMIT 10;
```

The four systems expose the levers of the previous section in different ways:

Lever	FAISS	Lucene	OpenSearch	pgvector
Fewer dimensions	PCA stage in the index string	truncation by the application	truncation by the application	index on a `subvector`
Fewer bits per vector	scalar, PQ, LSH	7, 4, or 1 bit per component	scalar, binary, PQ (FAISS engine)	`halfvec`, `binary_quantize`
Fewer vectors read	IVF, HNSW	HNSW per segment	HNSW, IVF (FAISS engine); shards	HNSW, IVFFlat
Exact search	`Flat` index	yes	yes	yes, without an index

Adding filters and keyword search

The vector query finds movies with mind-bending heist plots, but it ignores the rest of the request. We now add the filters on year and rating and the keyword part for the actor and the genre.

Three ways to filter

Combining a filter with a vector search is harder than it looks, because the ANN index knows nothing about the metadata. There are three strategies.

Pre-filtering applies the metadata condition first and then compares the query exactly with the vectors that pass. The result is exact, and the cost depends on how many vectors pass. A selective filter makes this cheap: if a filter admits 0.01% of the billion vectors of the previous section, an exact scan over the 100,000 survivors reads only 300 MB. A mild filter makes it expensive, because the scan over the survivors approaches a full scan.

Post-filtering runs the ANN search first and discards the results that fail the filter. This keeps the index intact, but it can return too few results. With HNSW and the default `hnsw.ef_search` of 40, a pgvector index scan delivers about 40 candidates. If the filter matches 10% of the rows, only about 4 of them survive, even if the query asked for 10. A fixed over-fetch, retrieving more candidates than requested, helps only if we guess the selectivity right. The robust form treats the index as a **stream of candidates** in order of increasing distance, like the incremental nearest-neighbor algorithm of Low-Dimensional Search Structures: the filter consumes candidates from the stream until k of them pass. HNSW suits this form well. Its search already maintains a queue of candidates, and continuing the search simply extends the queue into regions further away from the query. pgvector's iterative index scans work this way.

> **Post-filtering can silently return too few results**

> If a query asks for the ten nearest neighbors and then removes those failing a selective filter, the final list may contain far fewer than ten items, or be empty, even though many matching documents exist deeper in the ranking. Over-fetch a larger candidate set before filtering, and verify that enough results survive.

Filtering during the search evaluates the filter first, for example as a bitmap over all document ids, and passes it to the ANN search, which only accepts admitted documents as results. This fits the inverted file particularly well. An IVF search with PQ codes scans the lists of the probed centers entry by entry anyway; checking each entry against the bitmap costs one bit lookup and lets the scan skip the distance computation for every non-admitted vector. The search itself does not change, only its result is restricted. In the billion-vector example with 16 of 4,096 lists probed, the scan visits about 3.9 million codes. If the filter admits 1% of the collection, about 39,000 of these are admitted, far more than needed for a top 10.

The limit lies in very selective filters. If the filter admits only 0.001% of the collection, the probed lists contain about 39 admitted vectors, and the ten best of them may be poor matches, because the good admitted matches lie in lists that were not probed. Probing more lists helps, but at this selectivity the 10,000 admitted vectors fit into an exact scan of 30 MB, which is both cheaper and exact. Systems that filter during the search therefore choose per query: if only a few documents pass the filter, they skip the index and fall back to pre-filtering with an exact scan. Graph indexes can filter during the search as well, but they must still step through non-matching nodes, because these can lie on the path to a matching one. With a selective filter, most steps of the graph search are wasted.

Keyword search adds a second ranking. Building Semantic Search fused a BM25 ranking and a dense ranking with **reciprocal rank fusion** (RRF) in Pipeline D. RRF needs only ranks, so it fuses a cosine similarity and a BM25 score without calibrating them against each other. The four systems differ in which strategy they use for filters and in how much of the fusion they provide.

FAISS

FAISS has no metadata, so the application evaluates the filter on its own data and passes the admitted ids to the search. An `IDSelector` makes the index skip all other ids while it scans the probed lists, which is filtering during the search. With a selective filter, the probed lists may contain fewer than ten admitted movies, so it pays to raise n_{probe} or to switch to an exact scan over the admitted vectors. FAISS has no keyword search either. A separate engine computes the BM25 ranking, and the application fuses the two rankings:

```
# 1. Filter in the application, then search only the admitted ids
admitted = movies.query("year > 2005 and rating >= 7.5")["id"].to_numpy()
params = faiss.SearchParametersIVF(sel=faiss.IDSelectorBatch(admitted), nprobe=32)
_, dense_ids = index.search(q, 50, params=params)

# 2. BM25 ranking from a keyword engine, restricted to the same ids
sparse_ids = bm25_search('actors:"leonardo dicaprio" genres:thriller', admitted, 50)

# 3. Reciprocal rank fusion of the two rankings
def rrf(*rankings, k=60):
    scores = {}
    for ranking in rankings:
        for rank, doc_id in enumerate(ranking, start=1):
            scores[doc_id] = scores.get(doc_id, 0.0) + 1.0 / (k + rank)
    return sorted(scores, key=scores.get, reverse=True)

top10 = rrf(dense_ids[0], sparse_ids)[:10]
```

Lucene

Lucene filters during the search. The filter is passed as the fourth argument of `KnnFloatVectorQuery`. Lucene evaluates it per segment and searches the graph only for admitted documents. If fewer documents

pass the filter than the query asks for, it runs an exact search instead, and if the graph search visits too many vectors without completing, it stops and switches to an exact search as well. The keyword part becomes ordinary BM25 clauses of a Boolean query:

```
// 1. Filter on year and rating (FILTER clauses do not score)
Query filter = new BooleanQuery.Builder()
    .add(IntField.newRangeQuery("year", 2006, Integer.MAX_VALUE), Occur.FILTER)
    .add(FloatField.newRangeQuery("rating", 7.5f, Float.POSITIVE_INFINITY),
Occur.FILTER)
    .build();

// 2. Vector query that applies the filter during the graph search
Query knn = new KnnFloatVectorQuery("embedding", query, 50, filter);

// 3. Hybrid query: vector score + BM25 on actors and genres
Query hybrid = new BooleanQuery.Builder()
    .add(knn, Occur.SHOULD)
    .add(new PhraseQuery("actors", "leonardo", "dicaprio"), Occur.SHOULD)
    .add(new TermQuery(new Term("genres", "thriller")), Occur.SHOULD)
    .add(filter, Occur.FILTER)
    .build();
TopDocs top = searcher.search(hybrid, 10);
```

Two details matter here. First, the filter belongs inside the vector query. Adding it only as a `FILTER` clause of the Boolean query, next to an unfiltered `KnnFloatVectorQuery`, runs the two parts independently: the vector query returns its top 50 over all movies, and the Boolean query intersects them with the filter. This is post-filtering, with all its risks. Second, a Boolean query adds the scores of its clauses. The vector clause contributes at most 1, while a BM25 clause for a rare name can contribute several points, so the keyword part tends to dominate. Lucene itself provides no rank fusion; applications either weight the clauses with boosts or run the two queries separately and fuse their rankings with RRF.

OpenSearch

OpenSearch passes a filter inside the `knn` clause to the engine, which then filters during the search in the same way as Lucene. For the keyword part, a `hybrid` query runs its sub-queries separately, and a search pipeline fuses their results. Since version 2.19, the `score-ranker-processor` implements RRF:

```
# 1. Search pipeline that fuses the sub-query rankings with RRF
client.transport.perform_request("PUT", "/_search/pipeline/rrf-pipeline", body={
    "phase_results_processors": [{"score-ranker-processor": {
        "combination": {"technique": "rrf", "rank_constant": 60}}}],
})

# 2. Hybrid query: BM25 on actors and genres, k-NN on the overview,
#    both restricted by the same filter on year and rating
filters = {"bool": {"filter": [
    {"range": {"year": {"gt": 2005}}},
    {"range": {"rating": {"gte": 7.5}}},
]}}
response = client.search(index="movies", params={"search_pipeline": "rrf-pipeline"},
    body={"size": 10, "query": {"hybrid": {
        "queries": [
            {"bool": {"should": [
                {"match_phrase": {"actors": "Leonardo DiCaprio"}},
                {"match": {"genres": "thriller"}},
            ]}},
            {"knn": {"embedding": {"vector": query, "k": 50}}},
        ],
        "filter": filters,
    }}})
```

The `filter` of the hybrid query applies to both sub-queries. Instead of RRF, the pipeline can also use a `normalization-processor`, which rescales the scores of each sub-query to a common range and then combines them with weights. This keeps more information than ranks alone, at the cost of choosing a normalization and the weights.

pgvector

In SQL, the filter is an ordinary `WHERE` clause. With an approximate index, PostgreSQL applies it after the index scan, which is post-filtering with the risk described above. Since version 0.8, pgvector's iterative index scans continue through the index until enough rows pass the filter. The keyword part uses PostgreSQL's full-text search from Practical Frameworks, and the fusion is written in SQL:

```sql
-- 1. Filtered vector search; continue the index scan until enough rows pass
SET hnsw.iterative_scan = relaxed_order;

-- 2. Hybrid search: dense and sparse rankings, fused with RRF (k = 60)
WITH dense AS (
  SELECT id, ROW_NUMBER() OVER (ORDER BY embedding <=> '[0.021, ...]') AS r
  FROM movies
  WHERE year > 2005 AND rating >= 7.5
  ORDER BY embedding <=> '[0.021, ...]'
  LIMIT 50
),
sparse AS (
  SELECT id, ROW_NUMBER() OVER (ORDER BY ts_rank(tsv, q) DESC) AS r
  FROM movies, to_tsquery('(leonardo <-> dicaprio) | thriller') AS q
  WHERE tsv @@ q AND year > 2005 AND rating >= 7.5
  ORDER BY ts_rank(tsv, q) DESC
  LIMIT 50
)
SELECT m.title,
       COALESCE(1.0 / (60 + d.r), 0) + COALESCE(1.0 / (60 + s.r), 0) AS rrf
FROM dense d
FULL OUTER JOIN sparse s ON d.id = s.id
JOIN movies m ON m.id = COALESCE(d.id, s.id)
ORDER BY rrf DESC
LIMIT 10;
```

Here the `tsv` column covers the actors and the genres in addition to the title and the overview. The `dense` part ranks the admitted movies by cosine distance, the `sparse` part ranks the movies that match the actor or the genre by `ts_rank`, and the final query adds the RRF contributions of both rankings. A movie found by only one of the two rankings receives a contribution of zero from the other. As noted in Practical Frameworks, `ts_rank` is not BM25, but RRF only uses the order of the sparse ranking, which makes the difference smaller.

Choosing a system

The four systems implement the same methods but serve different settings. **FAISS** gives the most control over the index and the highest speed, including on GPUs, but it handles only vectors: filters, keyword search, storage, and updates remain the application's job. **Lucene** keeps vectors, text, and metadata in one index and filters during the search, which suits Java applications that embed their search. **OpenSearch** adds distribution across many machines and a complete hybrid pipeline with filters and rank fusion, at the price of operating a cluster. **pgvector** keeps vectors next to the relational data, with SQL filters, joins, and transactions, and suits collections that fit on a single database server. Dedicated vector databases, such as Milvus, Qdrant, and Weaviate, follow the same patterns, and several of them appear in the benchmark of the previous section.

Whatever the system, the query of this section needs the same three ingredients: an approximate vector search for the meaning, a keyword search for names and terms, and filters for the hard constraints. What differs is how much of the combination the system does for us.

Hands-on: Vector search with FAISS, OpenSearch, and pgvector

Send the movie query of this section through FAISS, OpenSearch, and pgvector: vector search, the three filter strategies, and hybrid search with RRF, and see why the systems rank differently below the clear winner. Open notebook ->

Includes pre-run results: you can read through or download and experiment.

Hands-on: Vector search with Lucene

Index the same movies with a `KnnFloatVectorField`, filter inside and next to the vector query, read the score sum of a hybrid Boolean query with `explain`, fuse with RRF instead, and try Lucene's quantized vector formats (Java kernel). Open notebook ->

Includes pre-run results: you can read through or download and experiment.

Summary

Method Comparison

The chapter's search methods trade recall, speed, and memory in different ways. Exact methods guarantee the true neighbors; the approximate methods relax that guarantee and pull one of three levers: fewer dimensions, fewer bits per vector, or fewer vectors read.

Method	Core idea	Lever	Recall knob	Memory per vector	Best for
Flat (exact)	Scan every vector	None	None (exact)	Full vectors	Small collections, ground truth
R-tree / grid file	Prune by bounding regions	Fewer vectors read (exact)	None (exact)	Full vectors	Low-dimensional data only
PCA / Matryoshka	Keep the leading dimensions	Fewer dimensions	Target dimension	20 to 60% less	Correlated features, MRL embeddings
Scalar quantization / VA-file	Few bits per component	Fewer bits per vector	Bits per component b	4 to 8 times less	Fast scans; exact with refinement (VA-file)
Product quantization	Codebook per sub-vector	Fewer bits per vector	Sub-vectors m, code bits b	16 to 64 times less	Billion-scale, tight memory
LSH	Sign bits of random hyperplanes	Fewer bits per vector	Number of hyperplanes h	About 100 times less	Angular similarity, cheap Hamming distance
IVF	Cluster, scan nearest lists	Fewer vectors read	Probed lists n_{probe}	Full vectors or codes, plus centers	Large collections, tunable recall
HNSW	Greedy search on a layered graph	Fewer vectors read	Candidate list ef_{search}	Full vectors plus about 256 bytes of links	Low latency and high recall in memory

The levers are complementary rather than exclusive. A production index reduces the dimensions, selects a few lists or graph regions to read, encodes the vectors in those lists compactly, and reranks the best k' candidates with exact distances. For one billion 768-dimensional vectors, the stack `PCA384,IVF4096,PQ48,RFlat` reads about 190 MB per query instead of 3 TB. On the ann-benchmarks GloVe-100 dataset, the fastest methods answer 60 to 850 times more queries per second than a full scan, depending on the recall target. Recall against exact search is only part of the story: after reranking, retrievers with far lower recall reach almost the quality of exact search (see Key Takeaway 6).

Key Takeaways

1. Vectors come from learned embeddings or from perceptual features such as color histograms. They are compared either with a similarity score, where larger is better (dot product, cosine), or with a distance, where smaller is better (Euclidean, Manhattan). The comparison function belongs to the feature. Length normalization turns cosine similarity into a plain dot product, and on the unit sphere ranking by Euclidean distance equals ranking by cosine.
2. Classical spatial indexes (Voronoi diagram, grid file, R-tree) were built for two and three dimensions. The optimal algorithm of Hjaltason and Samet runs on all of them and needs only an optimistic

bound per region, so it also works for the dot product. Even in 2D, a nearest-neighbor search follows several paths, because it must prove that no closer point exists.

3. The curse of dimensionality is quantifiable. Distances grow like $\sqrt{d}$ while their spread stays constant, so the contrast between the nearest and the farthest point vanishes, and dot products of random vectors concentrate around zero. Once the expected nearest-neighbor distance exceeds the size of a leaf, even the optimal algorithm visits every leaf, and decades of R-tree variants did not change this.
4. Loss of contrast also weakens the meaning of "nearest". Real data still works because related items form clusters that stand out from the background, but within a cluster the exact nearest neighbor is just one of several good matches. Vector search often serves as a first stage whose candidates a reranker reorders, and even without a reranker users want good matches rather than the single best one. Both arguments justify trading exactness for speed.
5. Approximate search pulls three levers. Fewer dimensions (PCA, Matryoshka) is a modest lever that still reads every vector. Fewer bits per vector (scalar quantization, product quantization, LSH) makes every comparison cheaper; the compression errors are small compared to the gap between good matches and the background, and the VA-file shows that bounds on the quantized cells even keep the search exact. Fewer vectors read (IVF, HNSW) visits only the regions near the query. Combined, the levers cut the work per query by two to three orders of magnitude, but their errors compound, so their parameters must be tuned together, on end-to-end quality rather than on recall alone.
6. Recall against exact search understates the quality of approximate search. On TREC-COVID, retrievers that read hundreds of times less data than a full scan return only a quarter of the exact top 10, yet after reranking many come within a few points of exact search, because the documents they return instead are often just as relevant. A reranker repairs fine-filter errors, such as coarse PQ codes, but not coarse-filter errors, such as unprobed IVF lists, so the coarse filter should stay generous.
7. In practice, vector search lives inside libraries (FAISS, Lucene), search engines (OpenSearch), and databases (pgvector). Realistic queries combine a vector search for meaning, a keyword search for names and terms, and filters for hard constraints. Filters are applied before, after, or during the search: pre-filtering suits selective filters, post-filtering suits graph indexes that deliver a stream of candidates, and filtering during the search suits the list scans of an inverted file. Dense and sparse rankings are fused with reciprocal rank fusion, as in Building Semantic Search.

Key Formulas

Key Formula: Cosine Similarity and Normalization

$$\mathrm{sim}_{\cos}(\boldsymbol{q}, \boldsymbol{d}_i) = \frac{\boldsymbol{q} \cdot \boldsymbol{d}_i}{\| \boldsymbol{q} \| \| \boldsymbol{d}_i \|} = \hat{\boldsymbol{q}} \cdot \hat{\boldsymbol{d}} \tag{6.14}$$

Normalize vectors once, then compare them with a dot product.

Key Formula: Product Quantization Distance

$$\tilde{\delta}(\boldsymbol{q}, \boldsymbol{p})^2 = \sum_{j=1}^{m} \| \boldsymbol{q}_j - \boldsymbol{c}_j(\boldsymbol{p}) \|^2 \tag{6.15}$$

The exact query sub-vectors are compared with the codebook centers that represent the stored vector; each term is a look-up in a table computed once per query.

Key Formula: Recall@k

$$\mathrm{recall@}k = \frac{|\ \text{approximate top-}k \cap \text{exact top-}k\ |}{k} \tag{6.16}$$

The ann-benchmarks quality axis: how well the approximate search reproduces exact search. It understates what the user sees, which is measured with nDCG against relevance judgments after reranking.

Reciprocal rank fusion, introduced in Building Semantic Search, remains the standard way to blend the dense and sparse rankings of a hybrid query.

Exam focus

- Similarity scores ("larger is better") versus distances ("smaller is better"), why the comparison function belongs to the feature, and how normalization connects cosine, dot product, and Euclidean distance.
- Why dense embeddings cannot be pruned the way sparse term vectors can in an inverted file.
- How the optimal nearest-neighbor algorithm works, why it needs only an optimistic bound per region, and why it follows several paths even in 2D.
- Why exact nearest-neighbor search degrades to a full scan in high dimensions, and why the loss of contrast and the retriever-reranker pipeline make good matches sufficient.
- The three levers of approximate search, what each one reduces, and why compression errors rarely change which matches stand out. How product quantization and LSH encode vectors, and how IVF and HNSW limit the vectors read.
- Pre-filtering, post-filtering, and filtering during the search: which one fits which filter selectivity and which index, and the failure mode of post-filtering with a selective filter.
- Why recall against exact search understates end-to-end quality, and why a reranker repairs fine-filter errors but not coarse-filter errors.

Self-Check Questions

1. (Understand) Why does storing length-normalized vectors let a system use the dot product in place of cosine similarity?
2. (Understand) Name the three levers of approximate search. For each, give one method and state whether it reduces the number of vectors read or the cost of reading each one.
3. (Apply) A query $\boldsymbol{q} = (2, 7)$ meets an R-tree leaf with rectangle $[3, 5] \times [1, 4]$. Compute the lower bound of the Euclidean distance, and state when the optimal algorithm may skip this leaf.
4. (Understand) An R-tree performs well at $d = 4$ but no better than a full scan at $d = 100$. What happens to the contrast between near and far points as the dimension grows, and why does this defeat pruning?
5. (Apply) A collection stores 512-dimensional float vectors. Product quantization splits them into $m = 64$ sub-vectors with $b = 8$ bits per code. How many bytes does a vector need before and after encoding, and how many entries do the lookup tables of one query hold?
6. (Evaluate) A colleague insists that a retriever must return the exact nearest neighbors of the query embedding. Using the loss of contrast and the retriever-reranker pipeline, argue why approximate results are usually sufficient.
7. (Analyze) A query for the ten nearest neighbors that also match a rare metadata value returns only two results, though many matching documents exist. Which filtering strategy was used, and how would you fix it for an HNSW index and for an IVF index?
8. (Analyze) A user searches for "a space adventure with Harrison Ford from the 1980s". Split the query into the parts a vector search, a keyword search, and a filter should handle, and explain why no single method covers all of them.
9. (Evaluate) Given a fixed recall target of 0.95, describe how you would use an ann-benchmarks recall-throughput plot to choose between HNSW and IVF with product quantization, and which information the plot does not show.
10. (Analyze) A retriever reaches a recall@10 of only 0.3 against exact search, yet after reranking its top 50 the result is as good as with exact search. Explain how this is possible, and why IVF with a single probed list does not recover in the same way.

Test Your Knowledge

Take the Chapter 6 Quiz ->>

Further Reading

- Guttman, A. (1984). **R-trees: a dynamic index structure for spatial searching**. *Proceedings of ACM SIGMOD*. ACM. The original R-tree, the bounding-rectangle index whose search algorithm and overlap problem this chapter builds on.
- Hjaltason, G. R., & Samet, H. (1999). **Distance browsing in spatial databases**. *ACM Transactions on Database Systems*, 24(2), 265-318. PDF. The incremental nearest-neighbor algorithm of the low-dimensional section: a priority queue over regions and points that returns neighbors one by one in order of distance and visits no more regions than necessary. The same stream of candidates reappears in post-filtering.
- Ram, P., & Gray, A. G. (2012). **Maximum inner-product search using cone trees**. *Proceedings of ACM SIGKDD*, 931-939. arXiv preprint (titled *Maximum inner-product search using tree data-structures*). Derives the bounds that let tree indexes prune for the dot product, including the bound $\boldsymbol{q} \cdot \boldsymbol{c} + r \parallel \boldsymbol{q} \parallel$ for a ball with center $\boldsymbol{c}$ and radius r.
- Weber, R., Schek, H.-J., & Blott, S. (1998). **A quantitative analysis and performance study for similarity-search methods in high-dimensional spaces**. *Proceedings of VLDB*. PDF. Shows analytically that above a moderate dimension any partitioning index degrades to a scan, and introduces the VA-file as the response, the core argument behind this chapter's curse-of-dimensionality section and the scalar quantization of the approximate-search section.
- Weber, R. (2001). **Similarity search in high-dimensional vector spaces**. Doctoral thesis, ETH Zurich, Diss. ETH No. 13974. *DISDBIS*, 74, Akademische Verlagsgesellschaft. ETH Research Collection (record only; the full text is closed access). The extended treatment of the VLDB paper above: cost models for tree-based and scan-based similarity search in high-dimensional spaces, and the VA-file in depth.
- Beyer, K., Goldstein, J., Ramakrishnan, R., & Shaft, U. (1999). **When is "nearest neighbor" meaningful?** *Database Theory: ICDT'99*, LNCS 1540, 217-235. Technical report PDF. Proves that for broad classes of data the contrast between the nearest and the farthest neighbor vanishes as the dimension grows, the result behind this chapter's loss-of-contrast formula.
- Aggarwal, C. C., Hinneburg, A., & Keim, D. A. (2001). **On the surprising behavior of distance metrics in high-dimensional space**. *Proceedings of ICDT*, LNCS 1973. PDF. Explains why distance concentration hits some metrics harder than others and why lower-norm distances can behave better in high dimensions.
- Weber, R., & Böhm, K. (2000). **Trading quality for time with nearest-neighbor search**. *Advances in Database Technology: EDBT 2000*, LNCS 1777, 21-35. Springer. Turns the VA-file into an approximate method and measures how much query time a small, controlled loss of result quality saves, the trade at the heart of the approximate-search section.
- Indyk, P., & Motwani, R. (1998). **Approximate nearest neighbors: towards removing the curse of dimensionality**. *Proceedings of ACM STOC*, 604-613. PDF. Introduces locality-sensitive hashing and shows that approximate nearest-neighbor search escapes the curse of dimensionality that defeats exact search.
- Charikar, M. S. (2002). **Similarity estimation techniques from rounding algorithms**. *Proceedings of ACM STOC*, 380-388. PDF. Introduces hashing with random hyperplanes and shows that two vectors receive different bits with probability θ/π, the basis of LSH for cosine similarity.
- Malkov, Y. A., & Yashunin, D. A. (2020). **Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs**. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4), 824-836. arXiv preprint 2016. The HNSW method that underlies most modern vector databases.
- Jégou, H., Douze, M., & Schmid, C. (2011). **Product quantization for nearest neighbor search**. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1), 117-128. PDF. Introduces product quantization, the compact encoding that makes billion-scale vector search fit in memory.
- Voorhees, E., Alam, T., Bedrick, S., Demner-Fushman, D., Hersh, W. R., Lo, K., Roberts, K., Soboroff, I., & Wang, L. L. (2020). **TREC-COVID: constructing a pandemic information retrieval test collection**. *ACM SIGIR Forum*, 54(1), 1-12. arXiv preprint. The test collection behind the end-to-end benchmark of the approximate-search section, including why its pooled judgments miss documents that new retrievers surface.

Tools and benchmarks: FAISS (composite index library), Lucene (search library with HNSW vector fields), OpenSearch vector search (distributed k-NN and hybrid search), pgvector (PostgreSQL vector search), and ann-benchmarks (standard recall-throughput comparison).